

FARMXPRESS

SOLUCIONES AGRÍCOLAS A TU ALCANCE



DESARROLLADO POR:
MANUEL JESÚS FERIA SANTANA

ÍNDICE

1. Objetivo.....	4
2. Justificación.....	4
3. Tecnologías Usadas.....	5
3.1 Tecnologías Front-End.....	5
3.2 Tecnologías Back-End.....	5
3.3 Frameworks.....	6
3.4 Entornos de Desarrollo	6
4. Análisis	7
4.1 Diagrama de Casos de Uso	7
4.2 Diagramas de Secuencia	8
4.2.1 Insertar Alquiler	8
4.2.2 Insertar Pago	9
4.2.3 Insertar Reseña	10
4.2.4 Consultar Catálogo	11
4.2.5 Adquirir Suscripción	12
4.3 Diagrama Entidad-Relación	13
4.4 Diagrama de Tablas Relacionales	13
4.4 Diagrama de Clases	14
4.5 Tablas de la Base de Datos	15
4.6 Diagramas de Gant	19
4.6.1 Marzo – Abril	19
4.6.2 Abril – Mayo	20
4.6.3 Mayo – Junio	21
4.6.4 Septiembre – Octubre	22
4.6.5 Octubre – Noviembre	23
4.6.6 Noviembre – Diciembre	24
4.7 Cumplimientos Legales	25
5. Guía de Estilos	26
5.1 Tipografía	26
5.2 Colores	26
5.3 Iconos	27
5.4 Imágenes	27

6. Desarrollo de la Aplicación	28
6.1 Programación del Lado Servidor	28
6.1.1 MVC (Modelo, Vista, Controlador)	28
6.1.3 PHPMAILER – Envío de Correos	33
6.1.2 Stripe – Pasarela de Pago	35
6.2 Programación del Lado Cliente	37
6.2.1 Validación Formulario de Registro	37
6.2.2 Validación Formulario de Producto	39
6.2.3 Formulario de Pago	39
6.2.4 Seguimiento de Productos	40
6.2.5 Generación de PDFs	42
6.2.6 Mensajes de Aviso	43
6.2.7 Creación de DataTables	45
6.2.8 Paginación de Contenido	46
7. Manual de Uso	49
7.1 Requisitos e Instalación	49
7.2 Usuario Invitado	49
7.3 Usuario Cliente	50
7.4 Usuario Proveedor	51
7.5 Administrador	51
8. Conclusiones	54
9. Recursos y Financiación.....	54
10. Futuro de la Aplicación	55
11. Enlaces Externos	55

1. OBJETIVO

El objetivo de esta aplicación, diseñada como empresa privada, es ayudar a los agricultores españoles facilitando una plataforma sencilla de usar en la que poder alquilar maquinaria agrícola de forma rápida y fácil, con la posibilidad de poder registrar a los proveedores de estas maquinarias que subirán fichas de sus productos a la propia web.

Los clientes podrán acceder a todo el catálogo de productos, marcar como favoritos los productos en los que estén interesados para su seguimiento, y dejar un comentario o reseña sobre los productos que hayan alquilado. Además, tanto clientes como proveedores podrán adquirir suscripciones con las que disfrutar de ciertas ventajas. Por otro lado, el sitio web contará con una zona de administrador en la que poder gestionar usuarios, categorías, productos y las diferentes suscripciones.

La web está alojada en el hosting Infinityfree, en la siguiente dirección: farmxpress.infinityfree.me

2. JUSTIFICACIÓN

La creación de una web de renting de maquinaria agrícola se justifica por la ausencia de una plataforma centralizada y abierta que permita a múltiples empresas proveedoras ofrecer sus productos para alquiler.

Actualmente, no existe una página que funcione como marketplace especializado en maquinaria agrícola, donde diferentes proveedores puedan subir sus propias máquinas y los usuarios clientes puedan comparar, elegir y alquilar según sus necesidades. La mayoría de las webs disponibles están enfocadas en el renting de vehículos, como coches o furgonetas, lo cual deja desatendido un sector clave como el agrícola, que requiere soluciones específicas para sus tareas.

Además, dentro del resto de opciones existentes en el territorio español encontramos a **Kubota** y **Rentagric**. Kubota se centra en el alquiler de sus propios productos, mientras que Rentagric, aunque se acerca más al objetivo de esta aplicación conectando proveedores de maquinaria agrícola con los clientes que estén interesados en sus productos, no permite que sean los propios clientes los que alquilen la maquinaria. Por tanto, una web como esta cubriría una necesidad real, fomentaría la competencia, daría visibilidad a más marcas y facilitaría el acceso a herramientas modernas para pequeños y medianos agricultores.

3. TECNOLOGÍAS USADAS

3.1 TECNOLOGÍAS FRONT-END

3.1.1 HTML (HYPERTEXT MARKUP LANGUAGE): El lenguaje de marcas estándar para estructurar y organizar el contenido de una página web. Define elementos como encabezados, contenedores, textos y formularios.

3.1.2. CSS (CASCADING STYLE SHEETS): Lenguaje de estilos en cascada que define cómo se presenta y se diseña el contenido estructurado por HTML. Controla colores, fuentes, márgenes, tamaños... de los elementos en la página.

3.1.3. JS (JAVASCRIPT): Es un lenguaje de programación de lado cliente utilizado para agregar dinamismo y funcionalidad avanzada a las páginas web, como validaciones, animaciones y actualizaciones en tiempo real.

3.1.4. JQUERY: Es una biblioteca de JavaScript que simplifica el manejo del DOM, la manipulación de eventos, animaciones y peticiones AJAX.

3.1.5. AJAX (ASYNCHRONOUS JAVASCRIPT AND XML): Es una técnica que combina JavaScript con el uso de objetos XMLHttpRequest para realizar peticiones al servidor de forma asíncrona, como insertar o actualizar la base de datos.

4.1.6. PDF24: Una librería para JavaScript que permite la generación de archivos PDF de forma sencilla.

4.1.7. DATATABLES: Librería de Javascript que permite crear tablas de visualización de datos fáciles de utilizar que permiten paginar, ordenar y filtrar los datos.

3.2 TECNOLOGÍAS BACK-END

3.2.1 PHP (HYPERTEXT PREPROCESSOR): Es uno de los lenguajes de programación del lado del servidor más populares, diseñado para crear aplicaciones web dinámicas. Se utiliza para procesar formularios, interactuar con bases de datos, generar contenido HTML dinámico y manejar sesiones de usuario.

3.2.2 MYSQL : El motor de base de datos utilizado para esta aplicación.

3.2.3 PASARELA DE PAGO STRIPE: Un cliente oficial para trabajar con la API de Stripe. Facilita la integración de pagos, suscripciones y otros servicios financieros en aplicaciones PHP. Permite, además, simular la acción de pagos para comprobar su funcionamiento.

3.3 FRAMEWORKS

3.3.1 BOOTSTRAP: Es un framework de front-end que facilita la creación de sitios web responsivos y modernos. Proporciona un sistema de rejillas, componentes predefinidos (botones, formularios, barras de navegación, etc.) y utilidades CSS y JS para agilizar el diseño y desarrollo.

3.3.2 AGRICULTURE: Una plantilla creada mediante Bootstrap que cuya temática principal está relacionada con la agricultura y el campo, presentando el verde como color principal

<https://bootstrapmade.com/agriculture-bootstrap-website-template/>

3.3.3 ADMINLTE: Otra plantilla gratuita creada mediante Bootstrap. Proporciona componentes predefinidos, y varias plantillas con diferentes estilos para desarrollar rápidamente paneles administrativos.

3.4 ENTORNOS DE DESARROLLO

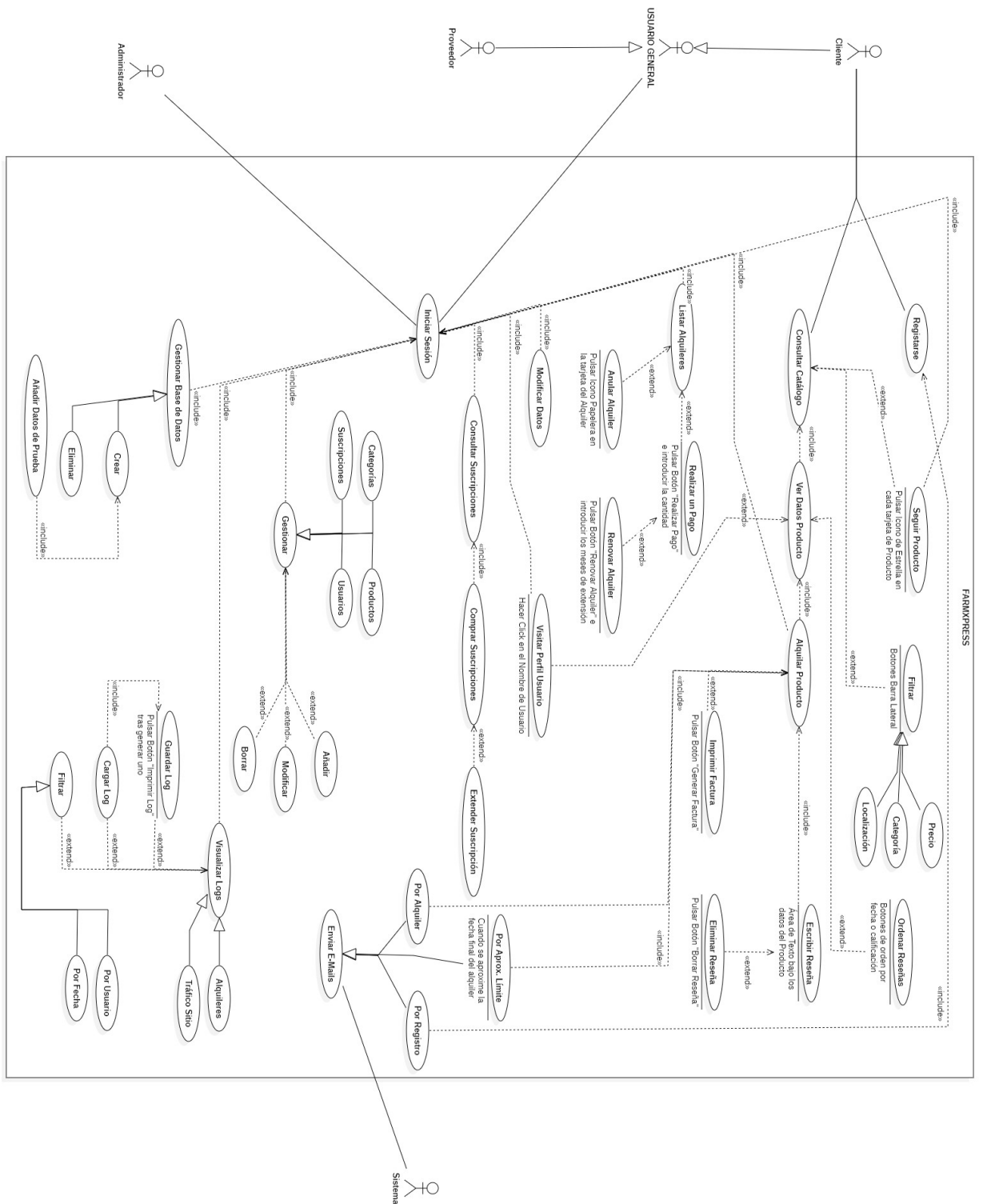
3.4.1 VISUAL STUDIO CODE (VSCODE): Es un editor de código fuente ligero y de código abierto desarrollado por Microsoft. Ofrece soporte para varios lenguajes de programación y una amplia gama de extensiones, como depuración, control de versiones y completado de código.

3.4.1 MODELO VISTA CONTROLADOR (MVC): Es un patrón de diseño de software utilizado en el desarrollo de aplicaciones web. Se divide en tres componentes:

- **Model (Modelo):** Maneja la lógica de los datos y la interacción con la base de datos.
- **View (Vista):** Se encarga de la presentación de los datos y la interfaz de usuario.
- **Controller (Controlador):** Gestiona la entrada del usuario y actualiza el modelo y la vista según sea necesario.

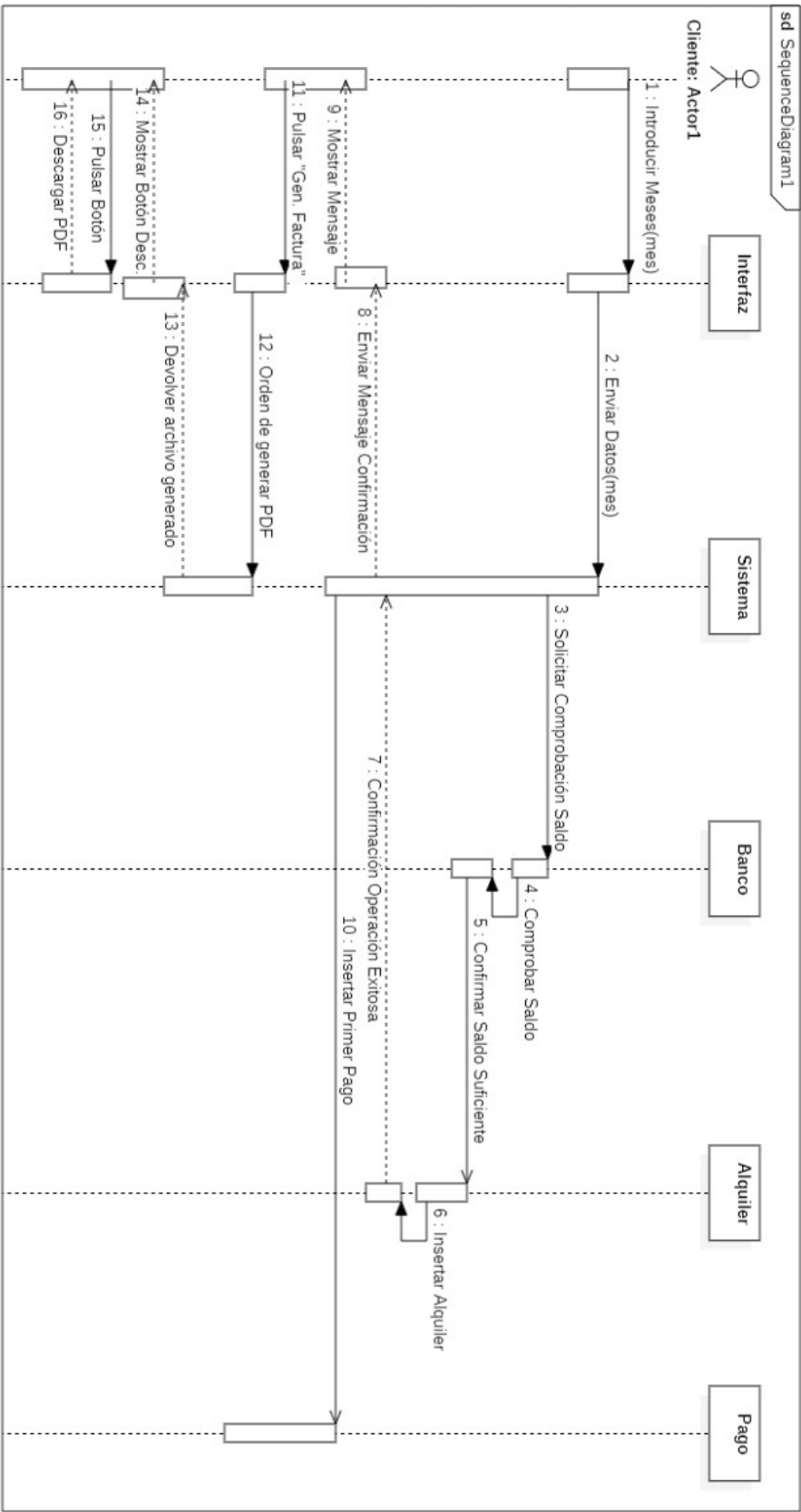
4. ANÁLISIS

4.1 DIAGRAMA DE CASOS DE USO

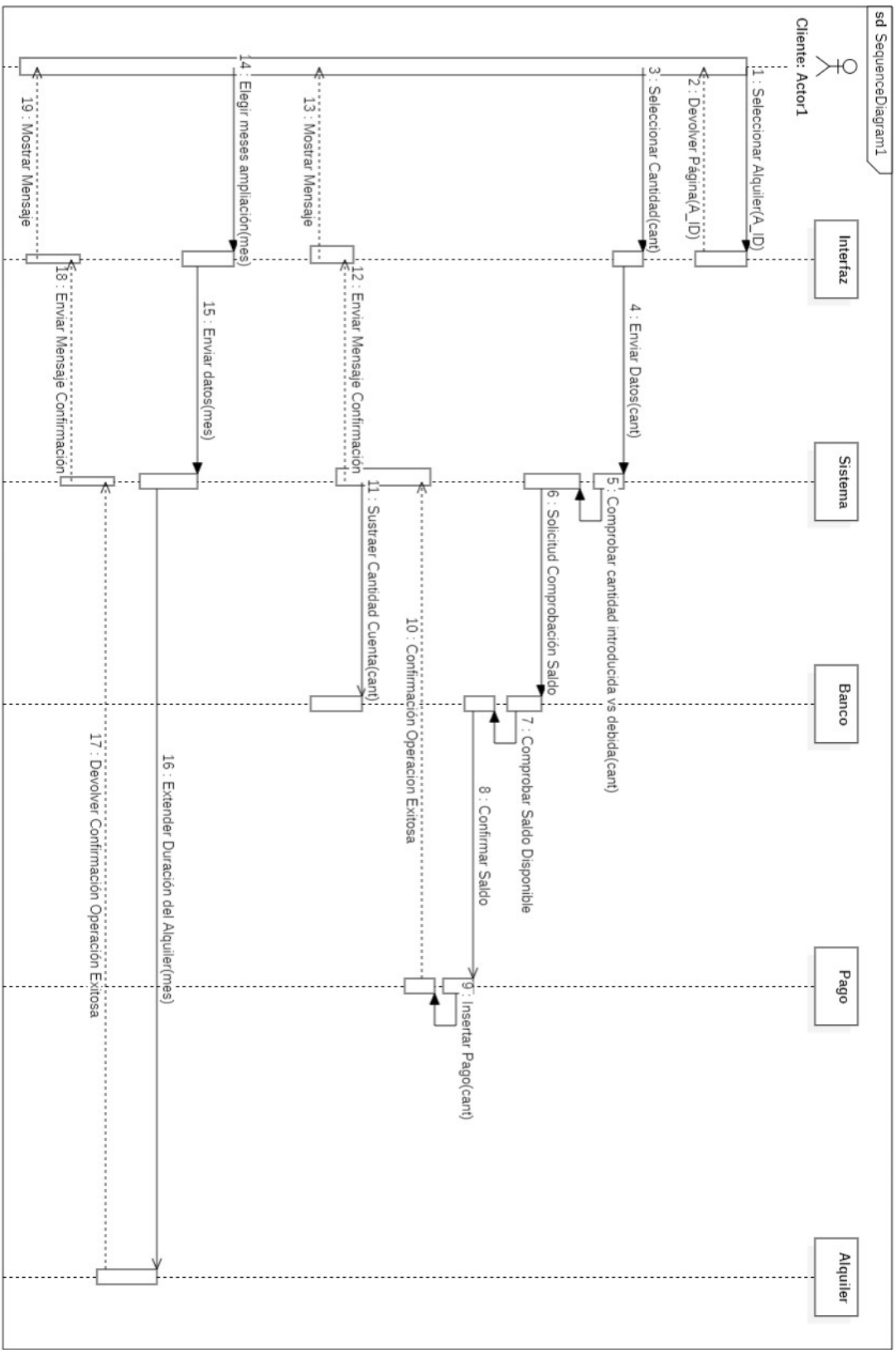


4.2 DIAGRAMAS DE SECUENCIA

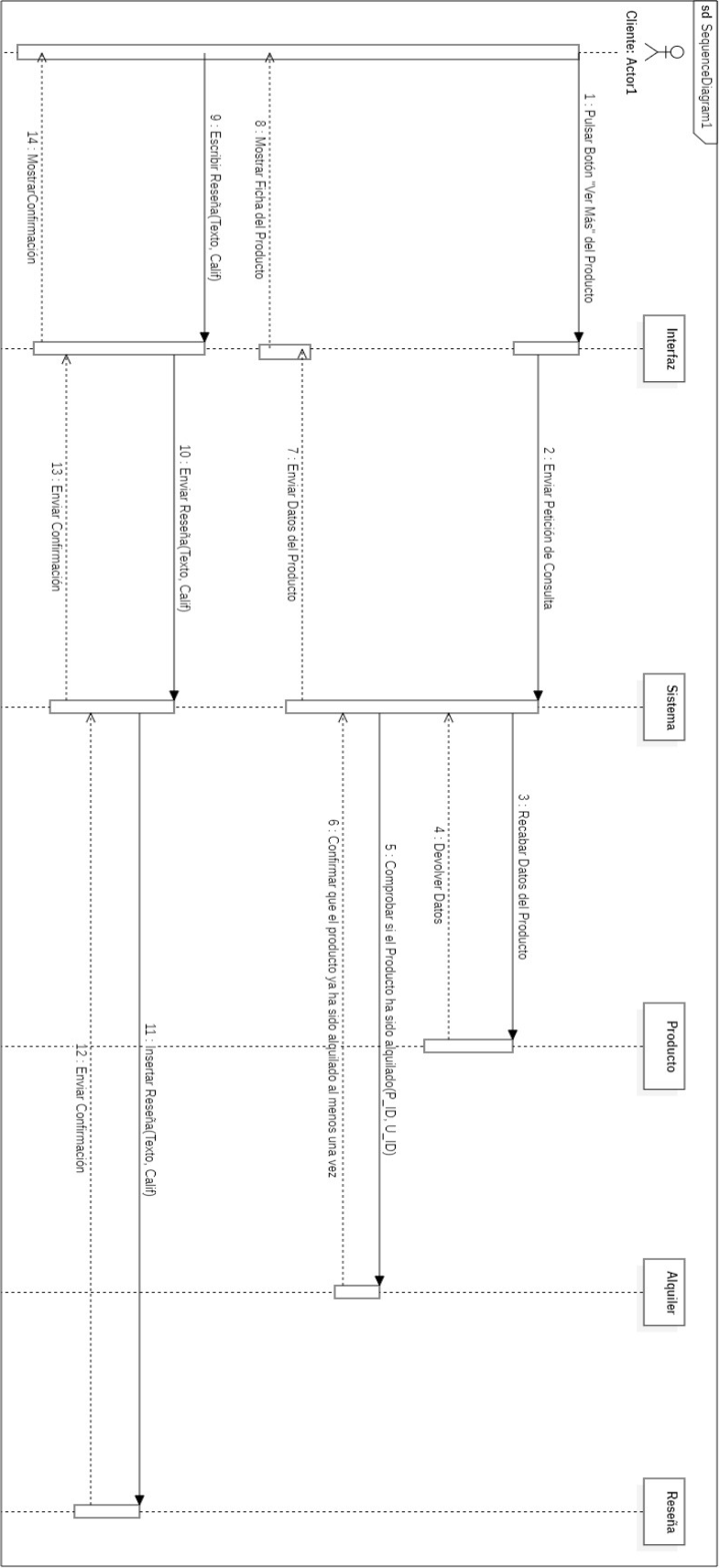
4.2.1 INSERTAR ALQUILER



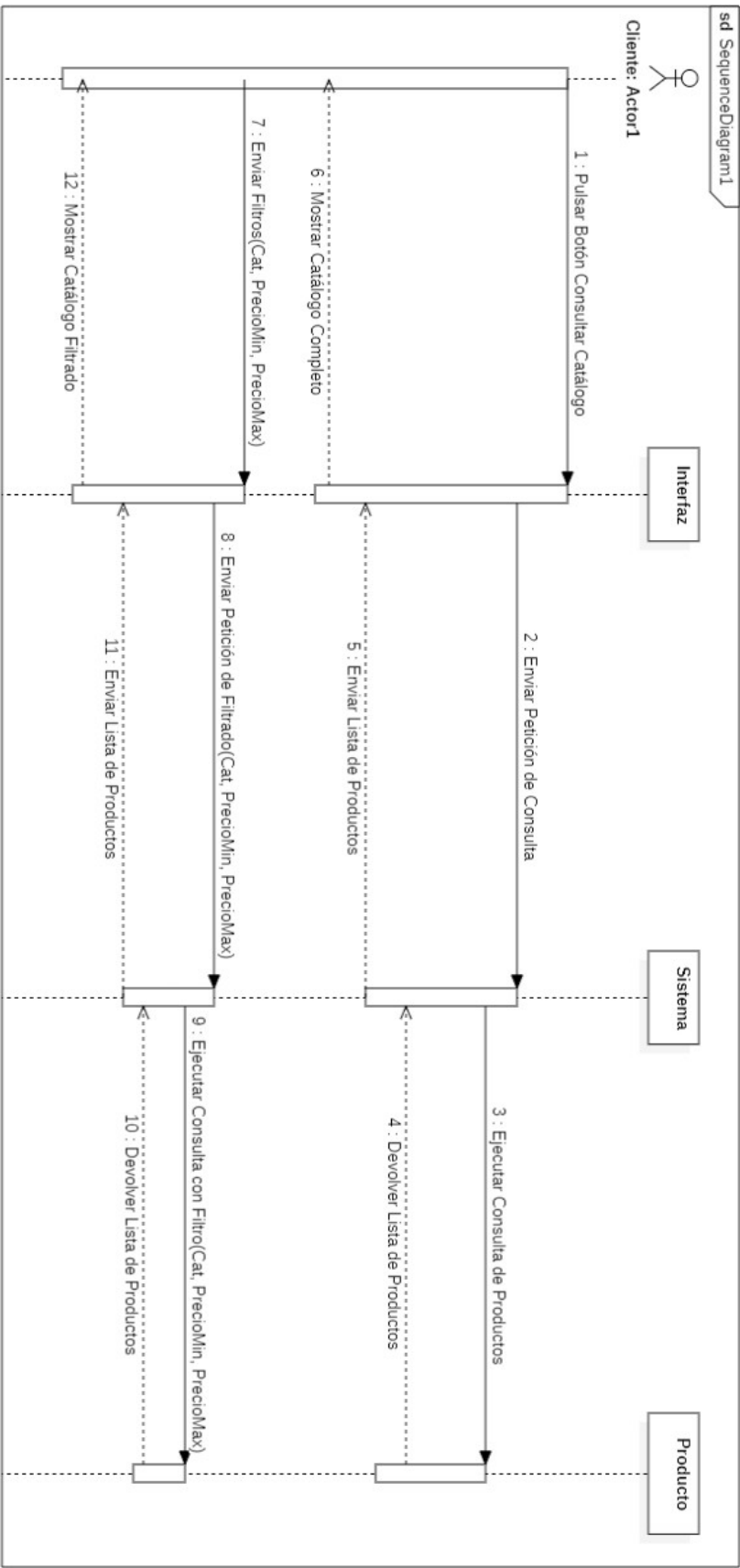
4.2.2 INSERTAR PAGO



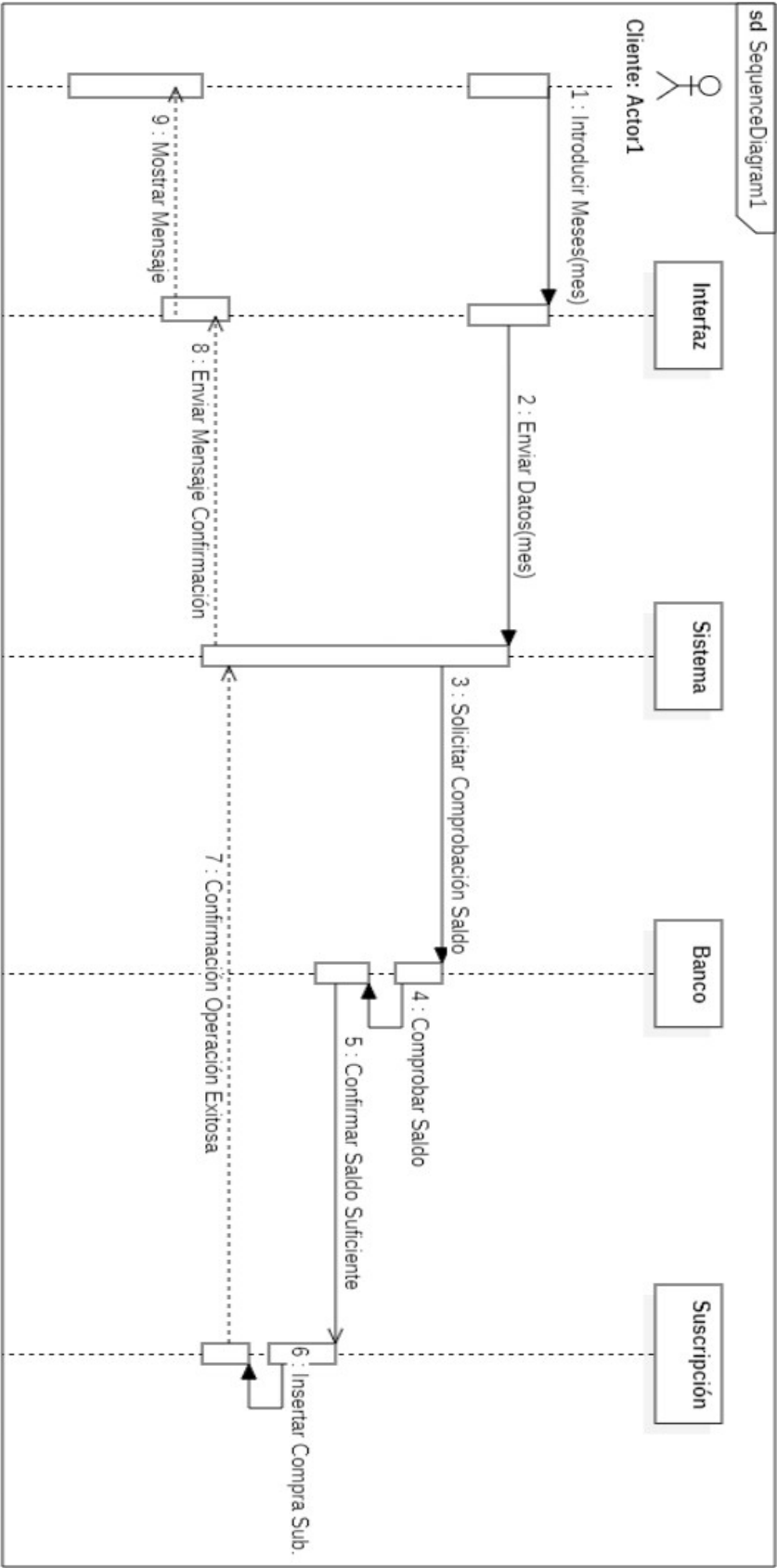
4.2.3 INSERTAR RESEÑA



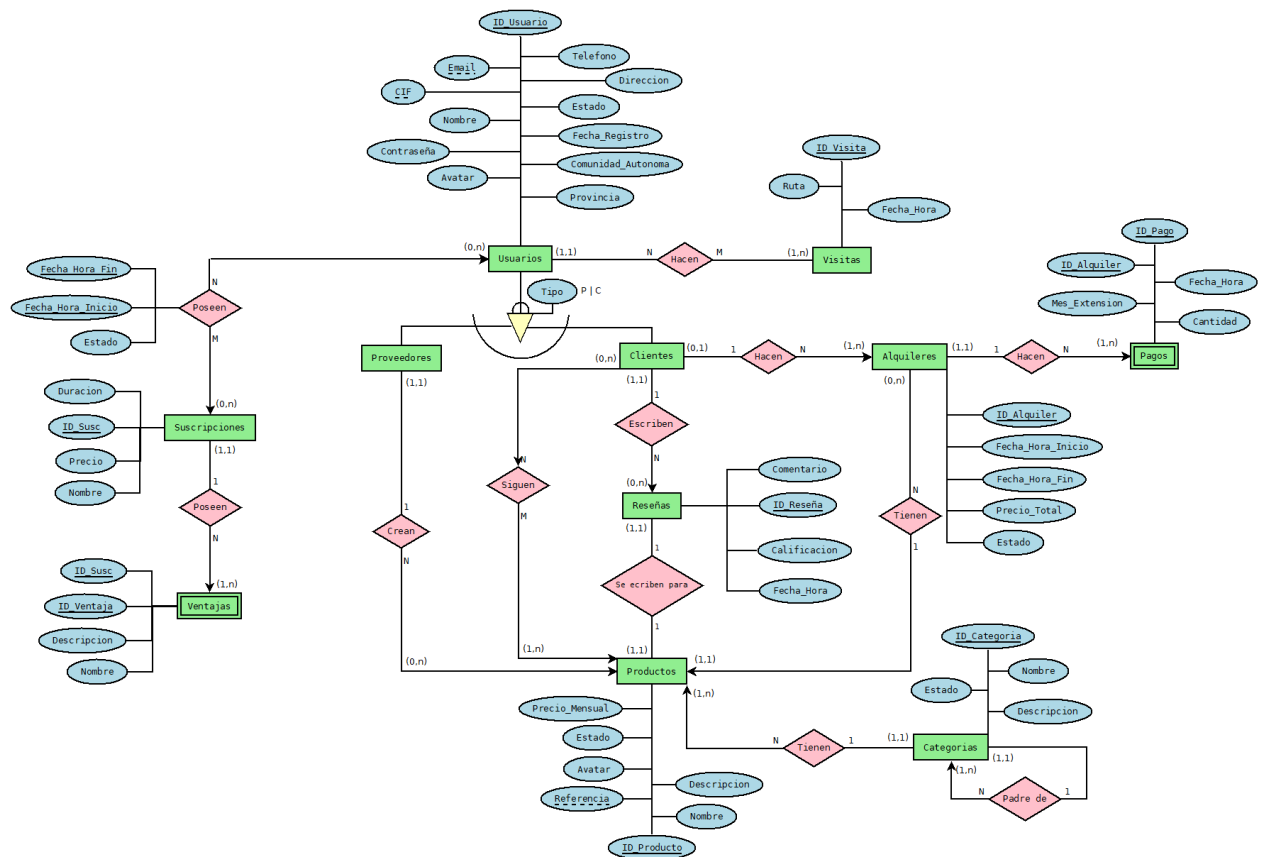
4.2.4 CONSULTAR CATÁLOGO DE PRODUCTOS



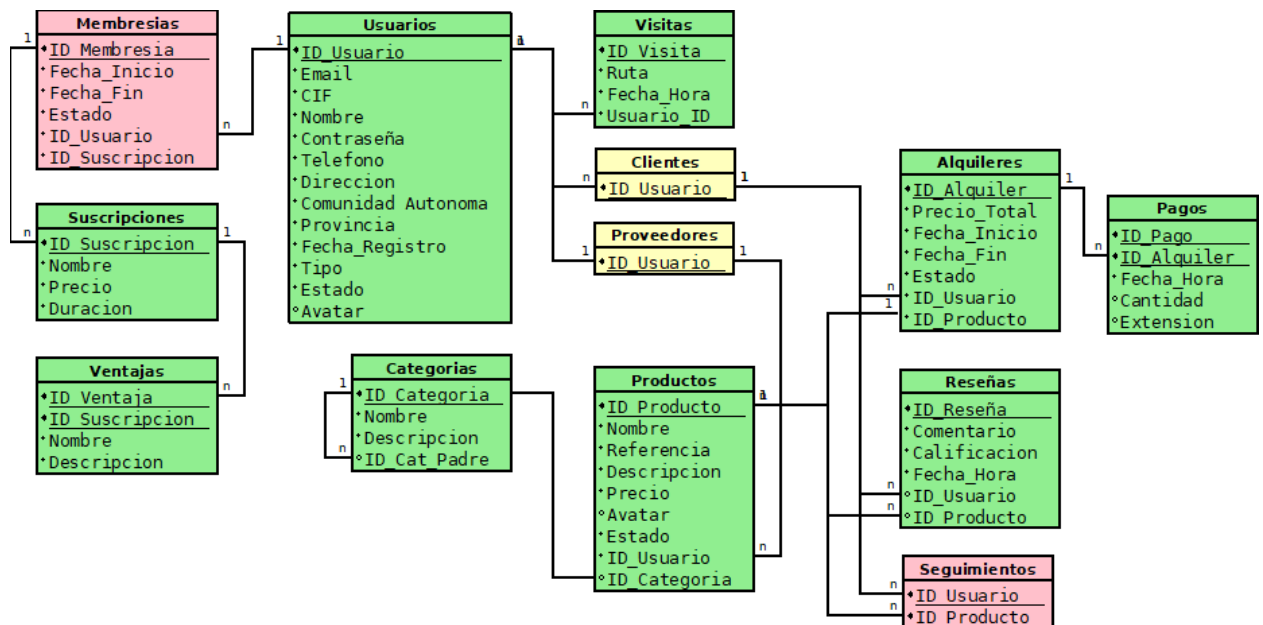
4.2.5 ADQUIRIR SUSCRIPCIÓN



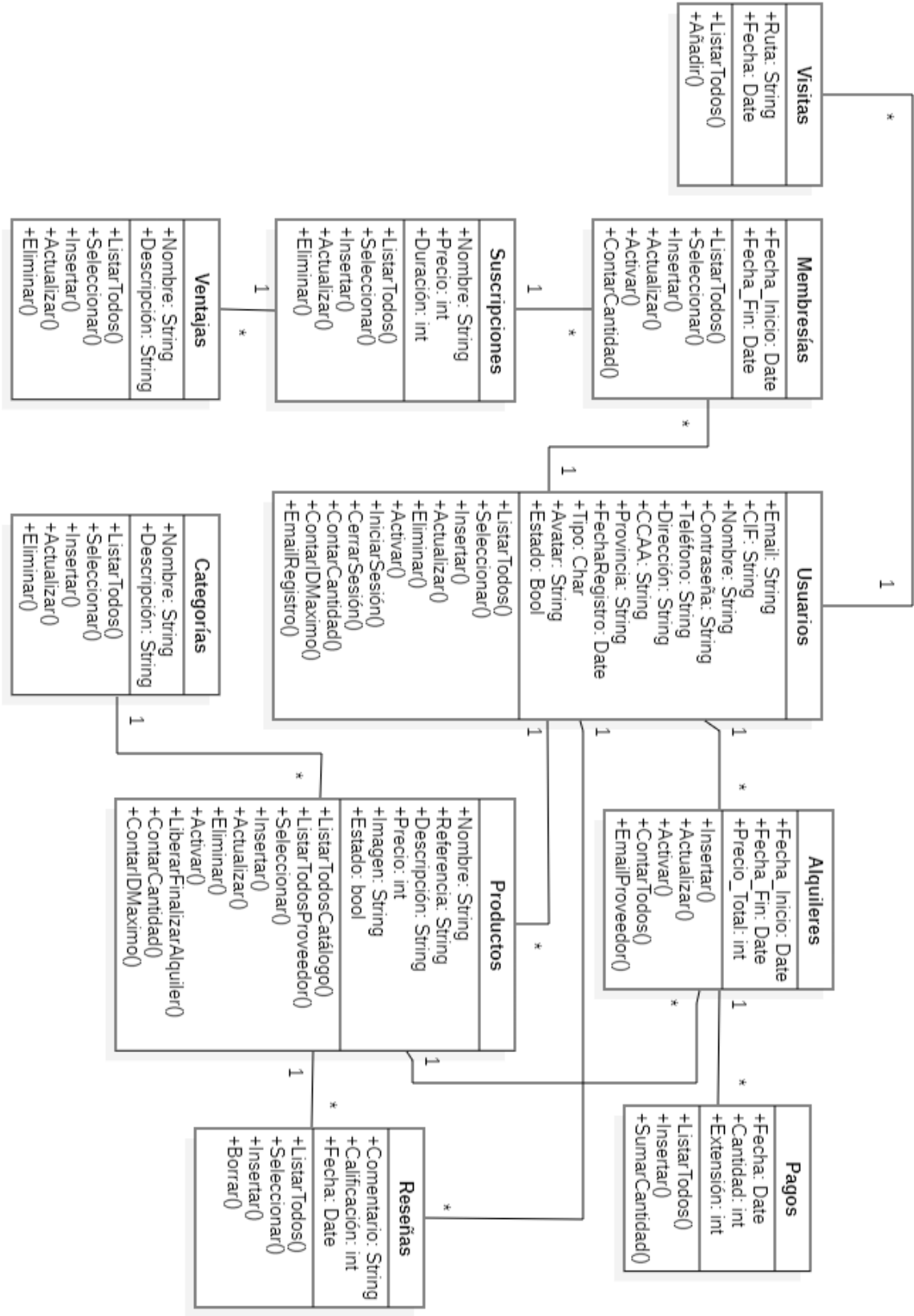
4.3 DIAGRAMA MODELO ENTIDAD-RELACIÓN (MER)



4.4 DIAGRAMA TABLAS RELACIONALES



4.5 DIAGRAMA DE CLASES



4.5 TABLAS DE LA BASE DE DATOS

4.5.1 USUARIOS

Información de los usuarios que se registran en la web.

USUARIO_ID	INT PRIMARY KEY	Identificador único del usuario
EMAIL	VARCHAR(100)	Correo electrónico único del usuario
CIF	VARCHAR(9)	Código CIF único de la empresa del usuario
NOMBRE	VARCHAR(100)	Nombre único de la empresa del usuario
CONTRASEÑA	VARCHAR(20)	Contraseña de la cuenta del usuario
DIRECCIÓN	VARCHAR(255)	Dirección física de la empresa del usuario
TELÉFONO	VARCHAR(9)	Teléfono único del usuario
TIPO	ENUM (P, C)	P para Proveedores, C para Clientes
AVATAR	BLOB	URL de la imagen de perfil del usuario
ESTADO	BOOL	1 = activo, 0 = inactivo
FECHA_REGISTRO	DATETIME	Fecha y hora en la que se registró el usuario

4.5.2 CLIENTES

Son los usuarios que alquilan productos y escriben reseñas sobre estos.

USUARIO_ID	INT PRIMARY KEY	Identificador del cliente asociado al campo USUARIO_ID

4.5.3 PROVEEDORES

Son los usuarios que suben sus propios productos a la web.

USUARIO_ID	INT PRIMARY KEY	Identificador del proveedor asociado al campo USUARIO_ID

4.5.4 VISITAS

Las páginas que visita cada usuario, para monitorear el flujo de navegación de la web.

VISITA_ID	INT PRIMARY KEY	ID de la visita a la web
RUTA	VARCHAR(255)	URL de la página visitada
FECHA_HORA	DATETIME	Fecha y Hora en la que se realizó la visita
USUARIO_ID	INT FOREIGN KEY	ID del usuario que entró en la página

4.5.5 SUSCRIPCIONES

Los usuarios pueden comprar una suscripción mensual y disfrutar de ciertas ventajas.

SUSCRIPCION_ID	INT PRIMARY KEY	Identificador de la suscripción
NOMBRE	VARCHAR(100)	Nombre de la suscripción
PRECIO_MENSUAL	DECIMAL(10, 2)	Precio de la suscripción mensualmente
DURACIÓN_BASE	INT	Meses que dura la suscripción al comprarla

4.5.6 VENTAJAS

Cada uno de los beneficios que otorgan las suscripciones.

VENTAJA_ID	INT PRIMARY KEY	ID de la ventaja
NOMBRE	VARCHAR(100)	Título de la ventaja
DESCRIPCIÓN	VARCHAR(255)	Descripción del beneficio que otorga la ventaja
SUSCRIPCION_ID	INT FOREIGN KEY	ID de la suscripción a la que pertenece

4.5.7 MEMBRESÍAS

Cada suscripción comprada por un usuario.

MEMBRESÍA_ID	INT PRIMARY KEY	Identificador de cada membresía
USUARIO_ID	INT FOREIGN KEY	ID del usuario que se ha suscrito.
SUSCRIPCION_ID	INT FOREIGN KEY	ID de la suscripción comprada por el usuario.
FECHA_INICIO	DATETIME	Fecha de compra.
FECHA_FIN	DATETIME	Fecha en la que se acabará la membresía
ESTADO	BOOL	1 = activa, 0 = terminada

4.5.8 CATEGORÍAS

Cada producto se puede clasificar según una categoría u otra.

CATEGORÍA_ID	INT PRIMARY KEY	ID de cada categoría
NOMBRE	VARCHAR(100)	Título de cada categoría
DESCRIPCIÓN	VARCHAR(255)	Descripción de cada categoría.
CAT_PADRE_ID	INT FOREIGN KEY	Cada categoría puede ser hija de otra.

4.5.8 PRODUCTOS

Cada proveedor sube productos a la web para que sean alquilados por los clientes

PRODUCTO_ID	INT PRIMARY KEY	ID del Producto
NOMBRE	VARCHAR(100)	Nombre del producto
DESCRIPCIÓN	VARCHAR(255)	Descripción de las características del producto.
REFERENCIA	CHAR(5)	Un código formado por una letra mayúscula y 4 números como identificador único.
PRECIO_MENSUAL	DECIMAL(10, 2)	Coste del alquiler del producto mensualmente.
IMAGEN	VARCHAR(255)	URL de la imagen del producto
ACTIVO	BOOL	1 = activo, 0 = inactivo o alquilado
USUARIO_ID	INT FOREIGN KEY	Proveedor dueño del producto
CATEGORÍA_ID	INT FOREIGN KEY	Categoría a la que pertenece el producto

4.5.9 RESEÑAS

Cada cliente que ha alquilado el producto en cuestión al menos una vez puede dejar un comentario sobre el producto en la página de éste, y valorarlo de 0 a 5 estrellas.

RESEÑA_ID	INT PRIMARY KEY	ID de la reseña o comentario
COMENTARIO	VARCHAR(255)	Texto de la reseña
CALIFICACIÓN	INT	Un número del 0 al 5 que indica el grado de satisfacción del cliente con el producto
FECHA_HORA	DATETIME	Fecha y Hora en la que se escribió la reseña
USUARIO_ID	INT FOREIGN KEY	Cliente que escribe la reseña
PRODUCTO_ID	INT FOREIGN KEY	Producto al cual se aplica la reseña

4.5.10 SEGUIMIENTOS

Cada usuario puede añadir o eliminar el producto que desee a una lista de “favoritos” o “seguimiento”, a modo de marcador para localizarlos fácilmente.

USUARIO_ID	INT FOREIGN KEY	ID del usuario interesado en el producto
PRODUCTO_ID	INT FOREIGN KEY	ID del producto añadido a seguimiento

4.5.11 ALQUILERES

Cada alquiler que un usuario realiza se almacena con los siguiente datos:

ALQUILER_ID	INT PRIMARY KEY	Identificador de cada alquiler
PRECIO_TOTAL	DECIMAL (10, 2)	Precio total del alquiler, se actualiza cada vez que el usuario decide renovarlo
FECHA_INICIO	DATETIME	Fecha en la que se realizó el alquiler
FECHA_FIN	DATETIME	Fecha de finalización del alquiler, se actualiza cada vez que el usuario decide renovarlo
ESTADO	BOOL	1 = activo, 0 = finalizado
USUARIO_ID	INT FOREIGN KEY	ID del Cliente que alquila el producto
PRODUCTO_ID	INT FOREIGN KEY	ID del Producto alquilado

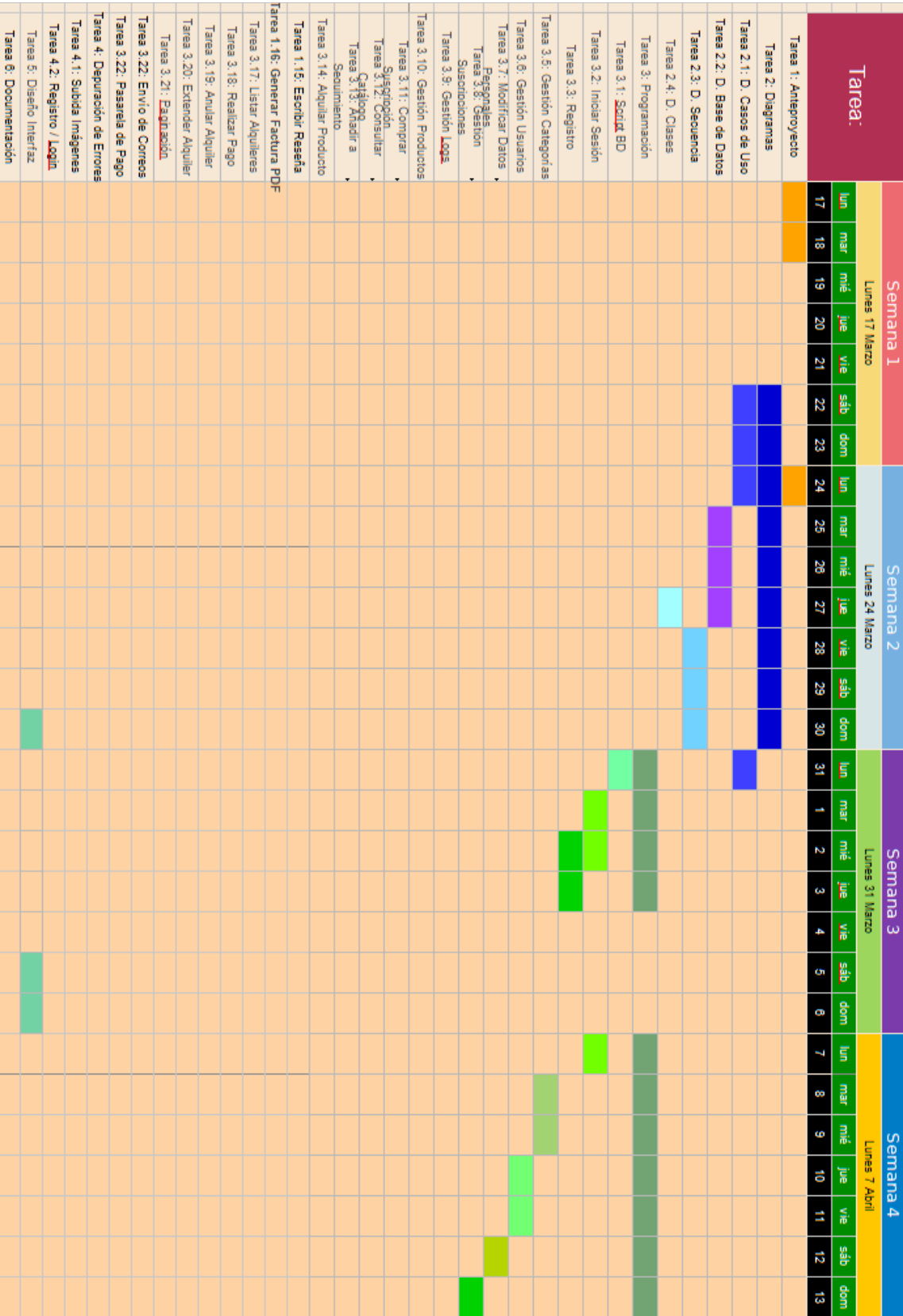
4.5.12 PAGOS

Cada vez que el usuario decide pagar el alquiler total o parcialmente y/o renovar el alquiler se almacena con los siguientes datos:

PAGO_ID	INT PRIMARY KEY	ID del pago realizado
CANTIDAD	INT	Dinero pagado en euros
MESES_EXTRA	INT	Meses que el usuario decide renovar el alquiler
FECHA_HORA	DATETIME	Fecha y Hora de realización del pago
ALQUILER_ID	INT FOREIGN KEY	ID del alquiler sobre el que se realiza el pago

4.6 DIAGRAMAS DE GANT

4.6.1 MARZO – ABRIL



4.6.2 ABRIL-MAYO

Tarea:	Semana 5							Semana 6							Semana 7							Semana 8						
	Lunes 14 de Abril							Lunes 21 de Abril							Lunes 28 de Abril							Lunes 5 de Mayo						
	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom
Tarea 1: Anteproyecto																												
Tarea 2: Diagramas																												
Tarea 2.1: D. Casos de Uso																												
Tarea 2.2: D. Base de Datos																												
Tarea 2.3: D. Secuencia																												
Tarea 2.4: D. Clases																												
Tarea 3: Programación																												
Tarea 3.1: Sciid BD																												
Tarea 3.2: Iniciar Sesión																												
Tarea 3.3: Registro																												
Tarea 3.5: Gestión Categorías																												
Tarea 3.6: Gestión Usuarios																												
Tarea 3.7: Modificar Datos																												
Personajes																												
Tarea 3.8: Gestión Suscripciones																												
Tarea 3.9: Gestión Logia																												
Tarea 3.10: Gestión Productos																												
Tarea 3.11: Compar																												
Suscripción																												
Tarea 3.12: Consultar																												
Catálogo																												
Tarea 3.13: Anadir a Seguimiento																												
Tarea 3.14: Alquiler Producto																												
Tarea 1.15: Escribir Reseña																												
Tarea 1.16: Generar Factura PDF																												
Tarea 3.17: Listar Alquileres																												
Tarea 3.18: Realizar Pago																												
Tarea 3.19: Anular Alquiler																												
Tarea 3.20: Extender Alquiler																												
Tarea 3.21: Paquidación																												
Tarea 3.22: Envío de Correos																												
Tarea 4: Depuración de Errores																												
Tarea 4.1: Subida Imágenes																												
Tarea 4.2: Registro / Login																												
Tarea 5: Diseño Interfaz																												
Tarea 6: Documentación																												

4.6.2 MAYO-JUNIO

Tarea:	Semana 9							Semana 10							Semana 11							Semana 12							
	Lunes 12 de Mayo							Lunes 19 de Mayo							Lunes 26 de Mayo							Lunes 2 de Junio							
	lun	mar	mié	jue	vie	sáb	dóm	lun	mar	mié	jue	vie	sáb	dóm	lun	mar	mié	jue	vie	sáb	dóm	1	2	3	4	5	6	7	8
Tarea 1: Anteproyecto																													
Tarea 2: Diagramas																													
Tarea 2.1: D. Casos de Uso																													
Tarea 2.2: D. Base de Datos																													
Tarea 2.3: D. Secuencia																													
Tarea 2.4: D. Clases																													
Tarea 3: Programación																													
Tarea 3.1: <u>Scrit</u> BD																													
Tarea 3.2: Iniciar Sesión																													
Tarea 3.3: Registro																													
Tarea 3.5: Gestión Categorías																													
Tarea 3.6: Gestión Usuarios																													
Tarea 3.7: Modificar Datos																													
Pagos																													
Tarea 3.8: Gestión Suscripciones																													
Tarea 3.9: Gestión Logs																													
Tarea 3.10: Gestión Productos																													
Tarea 3.11: Comprar																													
Suscripción																													
Tarea 3.12: Consultar																													
Gastos																													
Tarea 3.13: Asistir a Seguimiento																													
Tarea 3.14: Agregar Producto																													
Tarea 1.16: Escribir Reseña																													
Tarea 1.16: Generar Factura PDF																													
Tarea 3.17: Listar Alquileres																													
Tarea 3.18: Realizar Pago																													
Tarea 3.19: Anular Alquiler																													
Tarea 3.20: Extender Alquiler																													

4.6.3 SEPTIEMBRE-OCTUBRE

Tarea:	Semana 1							Semana 2							Semana 3							Semana 4						
	Lunes 22 Septiembre							Lunes 29 Septiembre							Lunes 6 Octubre							Lunes 13 Octubre						
	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom
Tarea 1: Administrar Rep. <u>Github</u>	22	23	24	25	26	27	28	29	30	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
Tarea 2: Administrar <u>Trello</u>																												
Tarea 3: Modificar Diagramas																												
Tarea 4: Registro Selector CCAA																												
Tarea 5: Filtro Productos CCAA																												
Tarea 6: Historial de Pagos																												
Tarea 7: Ordenar Reseñas																												
Tarea 8: Página Perfil Usuarios																												
Tarea 9: Implementar Filtros																												
Tarea 10: Envío Auto. <u>EMails</u>																												
Tarea 11: Vista <u>Estadísticas</u> Producto																												
Tarea 12: Mejorar Interfaz																												
Tarea 13: <u>Hostear</u> Proyecto																												
Tarea 14: Optimizar Código																												
Tarea 15: Validaciones <u>Backend</u>																												
Tarea 16: Depuración Errores																												
Tarea 17: Documentación																												

4.6.4 OCTUBRE-NOVIEMBRE

Tarea:	Semana 5							Semana 6							Semana 7							Semana 8								
	Lunes 20 Octubre							Lunes 27 Octubre							Lunes 3 Noviembre							Lunes 10 Noviembre								
	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Tarea 1: Administrar Rep. <u>Github</u>																														
Tarea 2: Administrar <u>Trello</u>																														
Tarea 3: Modificar Diagramas																														
Tarea 4: Registro Seleador <u>CCAA</u>																														
Tarea 5: Filtro Productos CCAA																														
Tarea 6: Historial de Pagos																														
Tarea 7: Ordenar Reseñas																														
Tarea 8: Página Perfil Usuarios																														
Tarea 9: Implementar Filtros																														
Tarea 10: Envío Auto. <u>E-Mails</u>																														
Tarea 11: Vista Estadísticas <u>Producto</u>																														
Tarea 12: Mejorar Interfaz																														
Tarea 13: <u>Hosteasr</u> Proyecto																														
Tarea 14: Optimizar Código																														
Tarea 15: Validaciones <u>Backend</u>																														
Tarea 16: Depuración Errores																														
Tarea 17: Documentación																														

4.6.5 NOVIEMBRE-DICIEMBRE

Tarea:	Semana 9							Semana 10							Semana 11						
	Lunes 17 Noviembre							Lunes 24 de Noviembre							Lunes 1 Diciembre						
	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom	lun	mar	mié	jue	vie	sáb	dom
Tarea 1: Administrar Rep. <u>Github</u>	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6
Tarea 2: Administrar <u>Trello</u>																					
Tarea 3: Modificar Diagramas																					
Tarea 4: Registro Seleccionador <u>CCAA</u>																					
Tarea 5: Filtro Productos CCAA																					
Tarea 6: Historial de Pagos																					
Tarea 7: Ordenar Reseñas																					
Tarea 8: Página Perfil Usuarios																					
Tarea 9: Implementar Filtros																					
Tarea 10: Envío Auto. <u>E-Mails</u>																					
Tarea 11: Vista Estadísticas <u>Producto</u>																					
Tarea 12: Mejorar Interfaz																					
Tarea 13: <u>Hostear</u> Proyecto																					
Tarea 14: Optimizar Código																					
Tarea 15: Validaciones <u>Backend</u>																					
Tarea 16: Depuración Errores																					
Tarea 17: Documentación																					

La implementación de cada tarea se ha realizado según su orden de importancia:

1. Planificación del proyecto: Borrador, diagrama de Casos de Uso, diagrama MER y Tablas relacionales, diagrama de Clases y diagramas de Secuencia.
2. Creación de la estructura de directorios y ficheros, incluir la plantilla Bootstrap inicial.
3. Implementación completa de la zona administrativa.
4. Implementación de las funciones de los proveedores.
5. Implementación de las funciones del Cliente.
6. Características extra durante el segundo período de trabajo de proyecto.

No ha sido posible implementar las funciones del Proveedor en primer lugar (añadir productos, principalmente) puesto que es el Administrador de la aplicación el único capaz de insertar Proveedores en la aplicación, además de las categorías a las que pueden pertenecer los productos. Y, del mismo modo, no ha sido posible programar las funciones de los Clientes ya que la mayoría de estas dependen de la existencia de productos insertados en la web.

Todas las tareas han sido realizadas por el Administrador del proyecto, pero en un equipo de trabajo se repartirían todas las tareas localizadas dentro del mismo nivel de importancia.

4.7 CUMPLIMIENTOS LEGALES

Se han incluido páginas legales en el pie de página de la web con información sobre el tratamiento de los datos de los usuarios que realiza esta aplicación, como Políticas de Privacidad o de Cookies. Además, se intentará cumplir con la integración de una nueva página, el Plan de Prevención Riesgos Laborales, recogiendo pautas sobre el uso prolongado de la pantalla, ergonomía...

El diseño de la interfaz pretende cumplir con los criterios de accesibilidad establecidos en las WCAG 2.1 (Web Content Accessibility Guidelines), permitiendo el uso de la plataforma por personas con diferentes capacidades. Para ello se intentará aplicar buenas prácticas como un buen contraste de colores o un mayor uso de etiquetas semánticas.



Pie de página de la aplicación con enlaces a páginas de interés

5. GUÍA DE ESTILOS

5.1 TIPOGRAFÍA

Para esta aplicación se utilizan dos tipos de fuente de letra principales:

- **OPEN SANS:** Para textos en el cuerpo de la aplicación.

Whereas disregard and contempt for
human rights have resulted

- **MARCELLUS:** Para botones de navegación y títulos.

Whereas disregard and contempt for
human rights have resulted

5.2 COLORES

Para esta web se utilizan los siguientes colores colores principales:

#116530 - #e0c9a6 Encabezado		#e0c9a6 - #8d5524 Pie de Página	
#116530 Color Principal		#2d465e Encabezados	
#fff8dc, #f4e2d8, #deb887 Cards de producto		#FFC1E3, #D6A4EC Cards de suscripciones	
Limegreen	Orange	Crimson	
Extender Suscripcion Ver y Crear Ventajas	Crear Pago Modificar Datos	Anular Suscripcion Eliminar Datos	

5.3 ICONOS

Se han utilizado algunos iconos gratis ofrecidos por la web FontAwesome, por ejemplo:

- **FA-GEAR** y **FA-TRASH**: Modificar y Eliminar Datos.
- **FA-PLUS**: Extender Alquiler, Añadir Ventajas a una suscripción (Admin).
- **FA-STAR**: Icono en el que pulsar para marcar o desmarcar productos favoritos.
- **FA-WHEAT-AWN**: Gestionar Productos.
- **FA-CREDIT-CARD**: Realizar Pago.
- **FA-RIGHT-TO-BRACKET**: Iniciar y Cerrar Sesión.

5.4 IMÁGENES

Logo de la aplicación e imagen utilizada de fondo en la zona de funcionalidades:



6. DESARROLLO DE LA APLICACIÓN

6.1 PROGRAMACIÓN DEL LADO SERVIDOR

6.1.1 MODELO VISTA CONTROLADOR (MVC)

Para el desarrollo óptimo y organizado de esta aplicación, se ha creado un controlador y un modelo para cada una de las tablas existentes de la base de datos, menos para Clientes y Proveedores, cuyas funciones se pueden incluir en el controlador y modelo de usuario al tener un único campo (USUARIO_ID).

Además, cada tabla también posee un “Índice” que es un documento que se incluirá en la página principal de la web al navegar por ella, que contiene las llamadas a las diferentes funciones que puede realizar cada Controlador.

En general, las funciones esenciales que contiene cada Controlador y Modelo son las siguientes, que corresponden a las funciones CRUD de una Base de Datos:

- **SELECCIÓN COMPLETA:** Seleccionar todas las filas existentes en la tabla, en muchos de ellos utilizando un parámetro llamado “\$offset” utilizado para la paginación del listado de la información recogida.
- **SELECCIÓN CONCRETA:** Seleccionar los datos de una única fila, para lo cual se utilizan los parámetros “\$field” y “\$value” (campo mediante el que buscar y valor de este campo).
- **INSERCIÓN:** Añadir los datos a su tabla correspondiente de la base de datos. En algunos casos, los datos se envían al modelo mediante una cookie codificada que los contiene, mientras que en otros, los datos se envían como parámetros directamente.
- **ACTUALIZACIÓN:** Modificar los datos de una fila en específico. Al igual que con las operaciones de Inserción, algunas se realizan enviando los datos al Modelo como una cookie codificada, pero en todos ellos se envía como parámetro la id del objeto a actualizar. No existe para algunos Modelos, como el correspondiente a la tabla “Pagos”, al no ser necesario.
- **ELIMINACIÓN:** Eliminar la fila seleccionada de la base de datos. Se pasa como parámetro el ID del objeto.
- **ACTIVACIÓN:** Para aquellas tablas que tienen un campo booleano “Estado”, sus modelos poseen un método que cambia el estado del objeto en cuestión (pasando su ID como parámetro).

```

/* Función: Actualizar un producto
 * Params: $id (ID del producto)
 * Return: Redirección a la página principal con éxito o error, o mensaje de éxito/error
 */
public function updateProduct(&$id){
    include("../assets/php/vBackEndProduct.php");
    $this->productModel->updateProduct($id, $data);
    header("Location: principal.php?methodProd=select&action=update");
}

/* Función: Actualizar un producto
 * Params: $id (ID del producto), $data (datos del producto)
 * Return: 1 en caso de éxito, -1 en caso de error
 */
public function updateProduct(&$id, &$data){
    try{
        $data[4]=($data[4]=="") ? null : $data[4];

        $sql=$this->db->prepare("UPDATE Productos SET Nombre=?, Descripción=?, Referencia=?,
                                Precio_Mensual=?, Categoría_ID=?, Imagen=? WHERE Producto_ID=?");
        $sql->bindValue(1, $data[0]);
        $sql->bindValue(2, $data[1]);
        $sql->bindValue(3, $data[2]);
        $sql->bindValue(4, $data[3]);
        $sql->bindValue(5, $data[4]);
        $sql->bindValue(6, $data[6]);
        $sql->bindValue(7, $id, PDO::PARAM_INT);
        $sql->execute();

        return 1;
    }catch(PDOException $e) {
        return -1;
    }
}

<?php
$data = array($_POST['name'], $_POST['desc'], $_POST['ref'], $_POST['price'], $_POST['cat'], null, null);
$allowedTypes = ['image/png', 'image/jpeg', 'image/jpg'];

if(preg_match('/^[a-zA-Z0-9À-ÿ\s]{3,100}$/', $data[0]))
    $data[0] = filter_var(trim($data[0]), FILTER_SANITIZE_FULL_SPECIAL_CHARS);
else header("Location: principal.php?methodProd=select?error=nombre");
if(preg_match('/^[a-zA-Z0-9À-ÿ\s,.-]{5,255}$/', $data[1]))
    $data[1] = filter_var(trim($data[1]), FILTER_SANITIZE_FULL_SPECIAL_CHARS);
else header("Location: principal.php?methodProd=select?error=descripción");
if(preg_match('/^[A-Z]{1}[0-9]{4}$/', $data[2]) && !is_array($this->selectProduct($data[2], $fields[2])))
    $data[2] = filter_var(trim($data[2]), FILTER_SANITIZE_FULL_SPECIAL_CHARS);
else header("Location: principal.php?methodProd=select?error=referencia");
if(is_numeric($data[3]) && $data[3] > 0)
    $data[3] = filter_var(trim($data[3]), FILTER_SANITIZE_NUMBER_FLOAT, FILTER_FLAG_ALLOW_FRACTION);
else header("Location: principal.php?methodProd=select?error=precio");

if(isset($_POST['prodId'])){
    $product=$this->selectProduct($_POST['prodId']);
    $data[6]=$product['Imagen'];

    if (isset($_FILES['imagen']) && $_FILES['imagen']['error'] === UPLOAD_ERR_OK) {
        if (in_array($_FILES['imagen']['type'], $allowedTypes) && $_FILES['imagen']['size'] <= (1024 * 1024)){
            $data[6]=$product['Producto_ID'].".strtolower(pathinfo($_FILES['imagen']['name'], PATHINFO_EXTENSION));
            $uploadFile = "../assets/img/products/".$data[6];
            move_uploaded_file($_FILES['imagen']['tmp_name'], $uploadFile);
        }else header("Location: principal.php?methodProd=select?error=imagen");
    }
}
else{
    if (isset($_FILES['imagen']) && $_FILES['imagen']['error'] === UPLOAD_ERR_OK) {
        if (in_array($_FILES['imagen']['type'], $allowedTypes) && $_FILES['imagen']['size'] <= (1024 * 1024)){
            $data[6]=($this->countProduct()['MAX']+1).".strtolower(pathinfo($_FILES['imagen']['name'], PATHINFO_EXTENSION));
            $uploadFile = "../assets/img/products/".$data[6];
            move_uploaded_file($_FILES['imagen']['tmp_name'], $uploadFile);
        }else header("Location: principal.php?methodProd=select?error=imagen");
    }
}
}
?>

```

*Ejemplo de actualización de un producto y Validación de datos en Entorno Servidor.
 Los datos enviados mediante el formulario de creación / edición de Productos son saneados.
 Para la subida de imágenes, se comprueba si existe la ID del producto (edición) o no.*


```

/* Función: Insertar un nuevo usuario
 * Params: No recibe parámetros
 * Return: Redirección a la vista de lista de usuarios o login según el tipo de usuario
 */
public function insertUser(){
    include("../assets/php/vBackEndUser.php");
    if($this->userModel->insertUser($data)==1){
        if(isset($_SESSION["User"]) && $_SESSION["User"]["Nombre"]!="ADMINISTRADOR"){
            $this->loginUser($data[2], $data[3]);
            $this->userModel->sendRegisterMail($data[0], $data[2]);
        }else header("Location: principal.php?methodUser=select&action=insert");
        }
    }
}

/* Funcion: Insertar un nuevo usuario
 * Params: $data (datos del usuario)
 * Return: 1 si se inserta correctamente, -1 en caso de error
 */
public function insertUser(&$data){
    try{
        $date=date("Y-m-d H:i:s");
        $estado=(isset($_SESSION["User"]) && $_SESSION["User"]["Nombre"]=="ADMINISTRADOR") ? 1 : 0;

        /*-----CONSULTA DE INSERTAR-----*/
        $sql=$this->db->prepare("INSERT INTO Usuarios (Email, CIF, Nombre, Contraseña, Telefono, Direccion, Comunidad,
        Provincia, Tipo, Fecha_Registro, Avatar, Estado) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)");
        for($i=0;$i<=8;$i++) $sql->bindValue($i+1, $data[$i]);
        $sql->bindValue(10, $date); //Fecha_Registro
        $sql->bindValue(11, $data[9]); //Avatar
        $sql->bindValue(12, $estado, PDO::PARAM_INT);
        $sql->execute();

        /*-----CLIENTE / PROVEEDOR-----*/
        $table = ($data[6]=="C") ? "CLIENTES" : "PROVEEDORES";
        $sql=$this->db->prepare("INSERT INTO $table (Usuario_ID) VALUES (?)");
        $sql->bindValue(1, $this->db->lastInsertId(), PDO::PARAM_INT);
        $sql->execute();

        /*-----CLIENTE / PROVEEDOR-----*/
        /*-----CONSULTA DE INSERTAR-----*/

        return 1;
    }catch(PDOException $e) {
        return -1;
    }
}

$data = array($_POST['email'], $_POST['cif'], $_POST['name'], $_POST['password'] ?? "PlaceholderPwd", $_POST['phone'],
    $_POST['address'], $_POST['region'], $_POST['province'], $_POST['tipo'] ?? "C", null);
$allowedTypes = ['image/png', 'image/jpeg', 'image/jpg'];

if(isset($_SESSION["usuario"]) && isset($_SESSION["usuario"]=="ADMINISTRADOR") $method="select";
else if(isset($_SESSION["usuario"]) && isset($_SESSION["usuario"]!="ADMINISTRADOR") $method="viewUpdate";
else $method="viewRegister";

if(filter_var($data[0], FILTER_VALIDATE_EMAIL) && !is_array($this->selectUser($data[0], $fields[0]))
    $data[0] = filter_var(trim($data[0]), FILTER_SANITIZE_EMAIL);
else header("Location: principal.php?methodUser=$method&error=email");
if(preg_match('/^[A-Z]{1}[0-9]{8}$/', $data[1]) && !is_array($this->selectUser($data[1], $fields[1]))
    $data[1] = filter_var(trim($data[1]), FILTER_SANITIZE_FULL_SPECIAL_CHARS);
else header("Location: principal.php?methodUser=$method&error=cif");
if(preg_match('/^[0-9a-zA-ZÀ-ÿ\s]{3,100}$/', $data[2]) && !is_array($this->selectUser($data[2], $fields[2]))
    $data[2] = filter_var(trim($data[2]), FILTER_SANITIZE_FULL_SPECIAL_CHARS);
else header("Location: principal.php?methodUser=$method&error=nombre");
if(preg_match('/^[0-9A-Za-z]{8,20}$/', $data[3])
    $data[3] = password_hash(filter_var(trim($data[3]), PASSWORD_BCRYPT);
else header("Location: principal.php?methodUser=$method&error=contraseña");
if(preg_match('/^[1-9]{1}[0-9]{8}$/', $data[4]) && !is_array($this->selectUser($data[4], $fields[4]))
    $data[4] = filter_var(trim($data[4]), FILTER_SANITIZE_FULL_SPECIAL_CHARS);
else header("Location: principal.php?methodUser=$method&error=telefono");
if(preg_match('/^[a-zA-Z0-9À-ÿ\s]{5,200}$/', $data[5]) && !is_array($this->selectUser($data[5], $fields[5]))
    $data[5] = filter_var(trim($data[5]), FILTER_SANITIZE_FULL_SPECIAL_CHARS);
else header("Location: principal.php?methodUser=$method&error=direccion");
if(is_string($data[6]) && is_string($data[7])){
    $data[6] = filter_var(trim($data[6]), FILTER_SANITIZE_STRING);
    $data[7] = filter_var(trim($data[7]), FILTER_SANITIZE_STRING);
}else header("Location: principal.php?methodUser=$method&error=provincia");

```

Ejemplo de inserción de Usuario y Validación de datos en Entorno Servidor.

Otros métodos que merece la pena mencionar son dos métodos incluidos en los Controladores de Alquileres y Membresías, que recogen todas ellas activas y comprueban los días restantes. Si faltan menos de cinco días para su caducidad, se envía un correo electrónico al usuario, y si ha superado la fecha de fin, se desactivan.

```

/* Función: Comprobar si hay alquileres que deberían desactivarse y notificar al usuario
 * Params: void
 * Return: Desactiva los alquileres caducados y notifica a los usuarios con alquileres próximos a caducar
 */
public function checkActiveRents(){
    $rentControl = $this->rentModel->listAllActiveRents();

    if(is_array($rentControl)){
        foreach($rentControl as $rent){
            $diffDays=(strtotime($rent["Fecha_Fin"]) - strtotime(date("Y-m-d")))/86400;

            if($diffDays<=0){ //Desactivar alquiler
                $this->activeRent($rent["Alquiler_ID"], $rent["Producto_ID"]);
            }else if($diffDays>0 && $diffDays<=5 && $rent["Mail_Bool"]==1){ //Notificar usuario
                global $dirAux; $mailType= 2;
                include($dirAux."_Indexes/Index_Product.php");
                include($dirAux."_Indexes/Index_User.php");

                $producto=$productController->selectProduct($rent["Producto_ID"]); //Recoger datos producto
                $usuario=$UserController->selectUser($rent["Usuario_ID"]); //Recoger datos cliente

                $this->rentModel->sendRentEmail($usuario["Email"], $producto["Nombre"], $mailType); //Enviar correo
                $this->toggleMailBoolRent($rent["Alquiler_ID"]); //Actualizar mail_bool a 0
            }
        }
    }
}

```

Método de desactivación automática de Alquileres

```

/* Función: Desactivar Membresía
 * Params: Ninguno
 * Return: Desactiva las membresías caducadas y notifica a los usuarios con membresías próximas a caducar
 */
public function activeMember(){
    $memberControl=$this->memberModel->listMember();

    if(is_array($memberControl)){
        foreach($memberControl as $mem){
            $diffDays=(strtotime($mem["Fecha_Fin"]) - strtotime(date("Y-m-d")))/86400;

            if($diffDays<=0){ //Desactivar membresía
                $this->memberModel->activeMember($mem["Membresía_ID"]);
                if(isset($_SESSION["User"]) && $mem["UNOM"]==$_SESSION["User"]["Nombre"]){
                    echo "<script>showBoxActiveMember('".$mem["SNOM"]."')</script>";
                }
            }else if($diffDays>0 && $diffDays<5){ //Enviar Email
                $this->memberModel->sendMemberEmail($mem["Email"], $mem["UNOM"], $mem["SNOM"]);
                $this->toggleMailBoolMember($mem["Membresía_ID"]);
            }
        }
    }
}

```

Método de desactivación automática de Membresías

```

/* Función: Desactivar un alquiler y reactivar producto
 * Params: $rId (ID del alquiler), $pId (ID del producto)
 * Return: Mensaje de éxito o error
 */
public function activeRent(&$rId, &$pId){
    global $dirAux; $mailType= 0; $active=1;
    include($dirAux."_Indexes/Index_Product.php");
    include($dirAux."_Indexes/Index_User.php");

    $producto=$productController->selectProduct($pId); //Recoger datos producto
    $usuario=$userController->selectUser($producto["Usuario_ID"]); //Recoger datos proveedor

    $rentControl = $this->rentModel->activeRent($rId); //Desactivar alquiler
    $productControl=$productController->activeProduct($pId, $active); //Reactivar Producto
    $this->rentModel->sendRentEmail($usuario["Email"], $producto["Nombre"], $mailType); //Enviar correo al proveedor
}

```

Nótese que esta función se ejecuta en cualquier página de la aplicación, por lo que una variable ya establecida en cada página llamada '\$dirAux' es necesaria para establecer la ruta correcta de los controladores y modelos

Como se ve en la imagen superior, la ejecución exitosa de la desactivación de un alquiler provoca la invocación de otro MVC y la llamada a uno de sus métodos. Esto es algo que también se da al insertar un Alquiler o al insertar un Pago:

```

/* Función: Insertar un nuevo alquiler junto al primer pago y desactivar el producto alquilado
 * Params: $pId (ID del producto), $month (meses de alquiler), $price (precio del alquiler)
 * Return: Mensaje de éxito o error
 */
public function insertRent(&$pId, &$month, &$price){
    if(isset($pId) && isset($month) && isset($price) && $pId!="" && $month!="" && $price!=""){ // Datos recibidos correctos
        include("_Indexes/Index_Product.php");
        include("_Indexes/Index_User.php");
        include("_Indexes/Index_Pay.php");
        $mailType= 1;
        $product = $productController -> selectProduct($pId); // Datos del Producto
        $user = $userController -> selectUser($product["Usuario_ID"]); // Datos del Proveedor del Producto

        if($product["Estado"]==1){
            if($this->rentModel->insertRent($pId, $month, $price)==1){ // Insertar Alquiler
                $rentID=$this->rentModel->db->lastInsertId(); // ID del Alquiler insertado
                $productController -> activeProduct($pId); // Desactivar Disponibilidad Producto

                if($payController->insertPay($rentID, $price, $month, $price)==1){ // Insertar Primer Pago
                    $this -> rentModel->sendRentEmail($user["Email"], $product["Nombre"], $mailType); // Enviar correo
                    $rentControl = $this->rentModel->selectRent($pId); // Datos del Alquiler para visualizar
                    include("Views/Client_Rent_Ok.php");

                    setcookie("data-rent", 0, time() - 3600, "/");
                }
            }else return "Error al alquilar producto.";
        }else header("Location: principal.php?methodProd=viewProduct&id=".$pId."&success=0");
        }else return "Error inesperado";
    }
}

```

Comprobar Estado del Producto > Insertar Alquiler > Desactivar Producto > Insertar Primer Pago (equivalente al primer mes de alquiler) > Enviar correo de notificación al Proveedor


```

/* Función: Insertar pago
 * Params: $rId (ID del alquiler), $mPrice (precio del mes),
 * $month (mes a pagar), $amount (cantidad a pagar, por defecto 0)
 * Return: Redirección a la página principal con éxito o error, o mensaje de éxito/error
 */
public function insertPay(&$rId, &$mPrice, &$month, &$amount=0){
    if($this->payModel->insertPay($rId, $month, $amount)==1){
        if($month!="" && !isset($_COOKIE["data-rent"])){
            include("_Indexes/Index_Rent.php");
            $rentController->updateRent($month, $mPrice, $rId);
        }

        if(isset($_GET["methodRent"])) return 1;
        else if(isset($_GET["methodPay"])){
            setcookie("data-pay", "", time() - 3600, "/");
            header("Location: principal.php?methodRent=select&page=1&success=1");
        }else
            return "Pago realizado correctamente";
    }else{
        if(isset($_GET["methodPay"]))
            header("Location: principal.php?methodRent=select&page=1&success=0");
        else
            return "Error al realizar el pago";
    }
}
}

```

Insertar Pago. Si no es el primer pago, el cual se inserta automáticamente, se llama a la función de actualización del alquiler en caso de que el usuario desee extenderlo.

Nótese las cookies, las cuales se crean durante el pago mediante Stripe Llamada al método de actualización del alquiler si el usuario desea extender su duración al realizar un pago.

6.1.2 PHPMailer – LIBRERÍA DE ENVÍO DE CORREOS

Gracias a PHPMailer, podemos enviar correos electrónicos automatizados desde un correo creado exclusivamente para la aplicación. Entre los correos que se envían, podemos observar:

1. Envío de correo de confirmación al registrarse:
2. Envío de correo al proveedor cuando uno de sus productos es alquilado:
3. Envío de correo al proveedor cuando uno de sus productos vuelve a estar disponible:
4. Envío de correo al cliente cuando uno de sus productos alquilados está a punto de caducar (5 días).
5. Envío de correo al administrador mediante el formulario de contacto de la página.

```

/* Funcion: Enviar un correo electronico al usuario
 * Params: $email (correo electronico del usuario), $name (nombre del usuario)
 * Return: Mensaje de exito o error al enviar el correo
 */
public function sendRegisterMail(&$email, &$name){
    try{
        require_once '../assets/vendor/PHPMailer/src/PHPMailer.php';
        require_once '../assets/vendor/PHPMailer/src/SMTP.php';
        require_once '../assets/vendor/PHPMailer/src/Exception.php';

        $phpmailer = new \PHPMailer\PHPMailer\PHPMailer;
        $phpmailer->isSMTP();
        $phpmailer->Host = 'smtp.gmail.com';
        $phpmailer->SMTPAuth = true;
        $phpmailer->SMTPSecure = \PHPMailer\PHPMailer\PHPMailer::ENCRYPTION_STARTTLS;
        $phpmailer->Port = 587;
        $phpmailer->Username = 'farmxpress@gmail.com';
        $phpmailer->Password = ' ';
        $phpmailer->setFrom('farmxpress@gmail.com', 'FarmXPress');
        $phpmailer->addAddress($email, $name);
        $phpmailer->isHTML(true);
        $phpmailer->Subject = 'Gracias por su registro';
        $phpmailer->Body = "Verifica su cuenta haciendo click
        <a href='https://localhost/FARMXPRESS/include/principal.php?methodUser=validate'>aquí</a>
        para poder comenzar a alquilar productos.";
        $phpmailer->SMTPOptions = array(
            'ssl' => array(
                'verify_peer' => false,
                'verify_peer_name' => false,
                'allow_self_signed' => true
            ));

        if($phpmailer->send()) return 1;
        else return 0;
    }catch(Exception $e) {
        return $e->getMessage();
    }
}

```

Función de envío de correo al usuario registrado. El enlace enviado ejecuta una función de actualización

```

/* Funcion: Enviar un correo electronico al usuario
 * Params: $email (correo electronico del usuario), $name (nombre del producto), $bool (tipo de correo)
 * Return: Mensaje de exito o error al enviar el correo
 */
public function sendRentEmail(&$email, &$name, &$bool){
    try{
        global $dirLocation; $dir = ($dirLocation == 0) ? "" : "../";
        require_once $dir.'assets/vendor/PHPMailer/src/PHPMailer.php';
        require_once $dir.'assets/vendor/PHPMailer/src/SMTP.php';
        require_once $dir.'assets/vendor/PHPMailer/src/Exception.php';

        //1=Nuevo Alquiler, 0=Alquiler Anulado//
        if($bool==1){
            $subject="Nuevo Alquiler";
            $body="Su producto $name ha sido alquilado.";
        }else if($bool==0){
            $subject="Uno de sus Productos vuelve a estar disponible";
            $body="Su producto $name vuelve a estar disponible al haber sido anulado su alquiler previo.";
        }else{
            $subject="Notificacion de Alquiler";
            $body="A su alquiler del producto $name le quedan solo 5 días.";
        }

        $phpmailer = new \PHPMailer\PHPMailer\PHPMailer;
        $phpmailer->isSMTP();
        $phpmailer->Host = 'smtp.gmail.com';
        $phpmailer->SMTPAuth = true;
        $phpmailer->SMTPSecure = \PHPMailer\PHPMailer\PHPMailer::ENCRYPTION_STARTTLS;
        $phpmailer->Port = 587;
        $phpmailer->Username = 'farmxpress@gmail.com';
        $phpmailer->Password = 'btrh raeq lpko zlxk';
        $phpmailer->setFrom('farmxpress@gmail.com', 'FarmXPress');
        $phpmailer->addAddress($email, $name);
        $phpmailer->isHTML(true);
        $phpmailer->Subject = $subject;
        $phpmailer->Body = $body;

        if($phpmailer->send()) return 1;
        else return 0;
    }
}

```

Función de envío de correo a los proveedores notificando sobre el alquiler o anulación de alquiler de uno de sus productos, o a los clientes si sus alquileres están a punto de caducar

6.1.3 STRIPE - PASARELA DE PAGO

Stripe es una plataforma que permite implementar funciones de pago electrónico de manera segura y eficiente. Una característica destacada de esta plataforma es su capacidad para simular pagos mediante su entorno de pruebas (sandbox). Esto permite a desarrolladores probar flujos completos de pago sin usar dinero real. Usando tarjetas de prueba y escenarios simulados, pueden verificar cómo responde el sistema ante transacciones exitosas o fallidas.

Para implementar esta pasarela de pago, en primer lugar se ha utilizado el siguiente comando para descargar la API oficial de Stripe dentro del proyecto:

composer require stripe/stripe-php

```
require '../assets/vendor/Stripe/vendor/autoload.php';
\Stripe\Stripe::setApiKey('secret_key_here');
```

Método de carga de la API de Stripe.

Nuestra clave secreta la obtendremos al crear una cuenta en la web oficial

```
/* Función: Crea una sesión de Stripe Checkout y redirige al usuario
 * Params: price (precio del producto)
 * Return: Redirección a Stripe Checkout
 */
try {
    $session = \Stripe\Checkout\Session::create([
        'payment_method_types' => ['card'],
        'line_items' => [[
            'price_data' => [
                'currency' => 'eur',
                'product_data' => ['name' => 'Operación de pago'],
                'unit_amount' => intval($price*100),
            ],
            'quantity' => 1,
        ]],
        'mode' => 'payment',
        'success_url' => $linkSuccess,
        'cancel_url' => 'http://localhost/FARMXPRESS/index.php',
    ]);

    // Redirect to Stripe Checkout
    header("Location: " . $session->url);
    exit();
} catch (Exception $e) {
    header("Location: http://localhost/FARMXPRESS/principal.php?methodProd=select&page=1&error=1");
}
```

Creación de sesión para la realización de pago

```

$linkSuccess="https://localhost/FARMXPRESS/include/";

/* Función: Recoge datos de pago y crea una sesión de Stripe Checkout
 * Params: pId (ID del producto), price (precio del producto), month (meses de alquiler)
 * Return: Cookie con los datos del alquiler o pago, redirige a Stripe Checkout
 */
if(isset($_GET['methodRent'])){
    $price= isset($_POST['price']) ? $_POST['price'] : 0;
    $cookie_array=setcookie("data-rent", json_encode([
        'pId' => isset($_POST['pId']) ? $_POST['pId'] : 0,
        'month' => isset($_POST['month']) ? $_POST['month'] : 0,
        'price' => $price
    ]), time() + (86400 * 30), "/");

    $linkSuccess.="principal.php?methodRent=insert";
}

}else if(isset($_GET['methodPay'])){
    $price= isset($_POST['amount']) ? $_POST['amount'] : 0;
    $cookie_array=setcookie("data-pay", json_encode([
        'rId' => isset($_POST['rId']) ? $_POST['rId'] : 0,
        'mPrice' => isset($_POST['mPrice']) ? $_POST['mPrice'] : 0,
        'month' => isset($_POST['month']) ? $_POST['month'] : 0,
        'amount' => $price,
    ]), time() + (86400 * 30), "/");

    $linkSuccess.="principal.php?methodPay=insert";
}

}else if(isset($_GET['methodMember'])){
    $data = json_decode($_COOKIE['data-member'], true);
    $price= isset($data['price']) ? $data['price'] : 0;

    $linkSuccess.="principal.php?methodMember=insert";
}

```

Manera en la que se manipulan los datos. Las cookies “data-rent” y “data-pay”, vistas anteriormente, se crean junto a la sesión de pago de Stripe Checkout.

← FarmXPRESS sandbox Sandbox

Operación de pago
€200.00

Pay with card

Email
email@example.com

Card information
1234 1234 1234 1234 VISA MasterCard American Express Discover
MM / YY CVC

Cardholder name
Full name on card

Country or region
Spain

☐ Securely save my information for 1-click checkout
Pay faster on FarmXPRESS sandbox and everywhere Link is accepted.

Pay

Ejemplo de visualización de Stripe Checkout para el alquiler de un producto que cuesta 200€ mensualmente

6.2 PROGRAMACIÓN DEL LADO CLIENTE

6.2.1 VALIDACIÓN DE DATOS DE FORMULARIO DE REGISTRO

Para el correcto funcionamiento de la web, los datos de los formularios de registro, inserción de productos, categorías... se comprueban y se validan mediante el uso de la librería JQuery de Javascript, que facilita la manipulación de los elementos del DOM de una página.

Para ello, se empieza seleccionando los valores de todos los campos de formulario, se recortan espacios con la función trim() y se declaran las expresiones regulares necesarias para el correcto formateo de los campos.

```
var email = $("#email").val().trim();
var cif = $("#cif").val().trim();
var name = $("#name").val().trim();
var phone = $("#phone").val().trim();
var address = $("#address").val().trim();
var password = "";
var region = $("#region").val().trim();
var file = $("#avatar").val().trim();

const emailRegex = /^[w._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,3}$/;
const cifRegex = /^[A-Z]{1}[0-9]{8}$/;
const nameRegex = /^[A-Za-z0-9\s]+$/;
const phoneRegex = /^[1-9][0-9]{8}$/;
const addressRegex = /^[A-Za-z0-9\s.,\-\#\@#\\\/]+$/;
const passwordRegex = /^[A-Za-z0-9\s.,\-\#\@#\\\/]+$/;
var province = $("#province").val().trim();

// Clear previous errors
$('#error-email, #error-name, #error-cif, #error-phone, #error-address, #error-region, #error-province, #error-pass
$('#email, #name, #cif, #phone, #address, #region, #province, #password, #avatar, #terms').removeClass('input-error

// Email validación
if(email.length < 5 || email.length > 100) {
    $('#error-email').text("El campo Email debe tener entre 5 y 100 caracteres.");
    $('#email').addClass('input-error').focus();
    return;
}
if (!emailRegex.test(email)) {
    $('#error-email').text("El campo Email no tiene un formato válido.");
    $('#email').addClass('input-error').focus();
    return;
}
let validEmail = await validateField("Email", email, "email", "#userId", "USUARIO");
if (!validEmail) return;
// Email validación
```

Ejemplo de validación del campo Email para el formulario de registro de usuario o de modificación de sus datos

Tras la declaración de variables, se comprueba si el dato introducido sigue correctamente la expresión regular declarada y unas normas de longitud. Al ser todos los campos de la tabla Usuario tipo Unique (menos la contraseña), se comprueba si ya hay otro usuario o existente con el dato en cuestión.

Para ello se utiliza la siguiente función asíncrona, validateField. Sus parámetros son:

- **Field:** Nombre del campo a comprobar en la tabla de Usuarios.
- **Value:** Valor el cual comprobar su existencia.
- **FieldName:** Nombre del campo del formulario, utilizado para vaciar su contenido si el dato ya existe.
- **idField:** id del objeto a modificar (campo invisible en del formulario).
- **Type:** Usuario o Producto, utilizado en la consulta.

```

async function validateField(field, value, fieldName, idField, type) {
    var id = (idField.length > 0) ? idField.val().trim() : null;

    return new Promise((resolve, reject) => {
        $.ajax({
            url: '../assets/php/validateField.php',
            type: 'POST',
            dataType: 'json',
            data: { field: field, value: value, objectId: id, type: type },
            success: function(response) {
                if(response.text != 0){
                    $("#error-"+fieldName).text("Ya hay un usuario registrado con este " + field);
                    $("#"+fieldName).addClass("input-error");
                    $("#"+fieldName).val("");
                    $("#"+fieldName).focus();
                    resolve(false);
                }else{
                    $("#"+fieldName).removeClass("input-error");
                    $("#error-"+fieldName).text("");
                    resolve(true);
                }
            },
            error: function(xhr, status, error) {
                $("#error").text("Error en la validación del campo " + field + ": " + error);
                reject(error);
            }
        });
    });
}

```

Esta función devolverá una Promesa, que se utiliza para marcar su llamada como “await”, es decir, que el programa tenga que esperar a su ejecución

```

<?php
$c = new PDO("mysql:host=localhost;dbname=FARMXPRESS", "root", "");
$c->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
header('Content-Type: application/json');

if($_POST['objectId']!=null)
    $query="SELECT * FROM ".$_POST["type"]."S WHERE ".$_POST["field"]." = :value AND ".$_POST["type"]."_ID != :objectId"
else
    $query="SELECT * FROM ".$_POST["type"]."S WHERE ".$_POST["field"]." = :value";

$sql= $c->prepare($query);
$sql->bindParam(':value', $_POST["value"]);
if($_POST['objectId']!=null)
    $sql->bindParam(':objectId', $_POST["objectId"]);

$sql->execute();
$result = $sql->fetchAll(PDO::FETCH_ASSOC);

if (is_array($result) && count($result) > 0)
    $result = $result[0][$_POST["field"]];
else $result = 0;

echo json_encode(["text" => $result]);
exit;
?>

```

Código de validación. Si existe una ID, se ignora la fila con esa ID, para no afectar a los campos que el usuario no desee modificar

Estos son los requisitos de validación de los datos del formulario de Usuario:

EMAIL	Máximo 100 caracteres, debe contener varios caracteres, un @ seguido de varios caracteres, un punto y 2-3 caracteres (ej: .com)
CIF	Una letra mayúscula y 8 dígitos
NOMBRE	Máximo 100 caracteres
CONTRASEÑA	Máximo 20 caracteres
TELÉFONO	9 dígitos
DIRECCIÓN	Máximo 255 caracteres
AVATAR	Formato .jpg o .png y máximo 1MB de tamaño

6.2.2 VALIDACIÓN DE DATOS DE FORMULARIO DE PRODUCTO

NOMBRE	Máximo 100 caracteres
DESCRIPCIÓN	Máximo 255 caracteres
REFERENCIA	Código formado por una letra mayúscula y una letra minúscula
PRECIO MENSUAL	Número positivo
IMAGEN	Imagen en formato .jpg o .png y máximo 1MB de tamaño

Sólo el campo Referencia es único, el cual también se comprueba de la misma forma, enviando el dato a un documento aparte que comprueba si la Referencia ya existe en la Base de Datos (ignorando la fila con la ID del producto en caso de modificar sus datos).

6.2.3 FORMULARIO DE PAGO

Para evitar que el usuario realizar un pago introduciendo una cantidad negativa, o una cantidad superior a la cantidad pendiente, se selecciona la suma total de los pagos hechos sobre ese alquiler mediante el siguiente método del Modelo de la entidad Pagos:

```

/* Función: Recuperar deudas de un alquiler
 * Params: $rId (ID del alquiler)
 * Return: Cantidad total adeudada, 0 si no hay pagos, -1 en caso de error
 */
public function selectDebtMoney(&$rId){
    try{
        $sql=$this->db->prepare("SELECT SUM(CANTIDAD) AS 'Debt_Money' FROM PAGOS WHERE ALQUILER_ID=?");
        $sql->bindValue(1, $rId, PDO::PARAM_INT);
        $sql->execute();

        if($sql->rowCount() != 0)
            return $sql->fetch(PDO::FETCH_ASSOC);
        else
            return 0;
    } catch(PDOException $e) {
        return -1;
    }
}

```


Tras recabar la cantidad, se resta al precio total del Alquiler y se inserta en un formulario oculto, que servirá para recabar ese valor mediante JQuery y realizar las comprobaciones adecuadas.

```
$("#amount").on("input", function() {  
    var value = $(this).val();  
    var maxAmount = parseInt($("#maxAmount").val());  
    if (value > maxAmount) $(this).val(maxAmount);  
    else if (value < 1 || isNaN(value)) $(this).val(1);  
});
```

Si la cantidad es mayor que la cantidad debida, el valor del formulario pasa a ser esa cantidad. Si el valor del formulario no es un número o es uno negativo, se vuelve 1

6.2.4 SEGUIMIENTO DE PRODUCTOS

```
function toggleFav(id, action){  
    $.ajax({  
        url: "principal.php?methodFav=toggleFav",  
        type: "POST",  
        data: {prodId:id, action:action},  
        success: function(data){  
            if(action=="add"){  
                showBoxActiveFav(1);  
                $("#add-"+id).attr("onclick", "toggleFav("+id+", 'del')");  
                $("#add-"+id).attr("id", "del-"+id);  
            }  
            else if (action=="del"){  
                showBoxActiveFav(0);  
                $("#del-"+id).attr("onclick", "toggleFav("+id+", 'add')");  
                $("#del-"+id).attr("id", "add-"+id);  
            }  
        },  
        error: function(){  
            alert("Error inesperado");  
        }  
    });  
  
    $("#fav-star-"+id).toggleClass("fa-regular");  
    $("#fav-star-"+id).toggleClass("fas");  
    event.preventDefault();  
}
```


La función superior se incluye en la página del catálogo de productos, solo presente si el usuario ha iniciado sesión, y es asignada a un botón con icono de estrella en cada tarjeta de producto. La operación envía tanto la acción como la ID al Modelo de “Favoritos”, y realiza una consulta de inserción o eliminación en la tabla de Seguimientos gracias al ID enviado.

```

/* Función: Insertar Producto en Favoritos o eliminarlo
 * Params: $id (ID del producto), $action (add para añadir, del para eliminar)
 * Return: 1 si la operación fue exitosa, -1 en caso de error
 */
public function toggleFav(&$id, &$action){
    try{
        if($action=="add")
            $query="INSERT INTO Seguimientos (Producto_ID, Usuario_ID) VALUES (?,
            (SELECT Usuario_ID FROM Usuarios WHERE Nombre=?))";
        else if($action=="del")
            $query="DELETE FROM Seguimientos WHERE Producto_ID=? AND Usuario_ID=
            (SELECT Usuario_ID FROM Usuarios WHERE Nombre=?)";

        $sql=$this->db->prepare($query);
        $sql->bindValue(1, $id, PDO::PARAM_INT);
        $sql->bindValue(2, $_SESSION["User"]["Nombre"], PDO::PARAM_STR);
        $sql->execute();

        return 1;
    }catch(PDOException $e) {
        return -1;
    }
}

```

La acción determina el tipo de consulta (Añadir o Eliminar)

En último lugar, todos tus productos añadidos a favoritos aparecerán al pulsar “Productos Favoritos” en la barra de navegación. Esta sección también contiene este script, para poder desmarcar los productos fácilmente.



6.2.5 GENERACIÓN DE PDFs

Al alquilar un producto, es posible generar una factura descargable en formato PDF con los datos del alquiler. La creación del PDF se realiza mediante la librería “PDF24”.

```
async function pdfCreate(){
    const rentDetails = document.getElementById("rent-details");
    const date = new Date();

    var archivoPDF = new PDF24Doc();
    archivoPDF.setCharset("UTF-8");
    archivoPDF.setFilename("FacturaFarmXpress.pdf");
    archivoPDF.setPageSize(210, 297);

    var contenidoPDF = new PDF24Element();
    contenidoPDF.setTitle("Factura de Alquiler");
    contenidoPDF.setUrl("http://www.pdf24.org");
    contenidoPDF.setAuthor("FarmXPress");
    contenidoPDF.setDateTime(date.getTime());
    var texto=rentDetails.innerHTML
    contenidoPDF.setBody(texto);
    archivoPDF.addElement(contenidoPDF);

    archivoPDF.create();
}

PDF24Doc.prototype.setPageSize = function(width, height) {
    this.setParam('pageSize', width + 'x' + height);
};
PDF24Doc.prototype.setEmailTo = function(emailAddr) {
    this.setParam('emailTo', emailAddr);
};
PDF24Doc.prototype.addEmailTo = function(emailAddr) {
    if(this.params.emailTo && this.params.emailTo != '') {
        this.params.emailTo += ';' + emailAddr;
    } else {
        this.params.emailTo = emailAddr;
    }
};
PDF24Doc.prototype.setEmailFrom = function(emailAddr) {
    this.setParam('emailFrom', emailAddr);
};
PDF24Doc.prototype.setEmailSubject = function(subject) {
    this.setParam('emailSubject', subject);
};
PDF24Doc.prototype.setEmailBodyType = function(bodyType) {
    this.setParam('emailBodyType', bodyType);
};
PDF24Doc.prototype.setEmailBody = function(body) {
    this.setParam('emailBody', body);
};
PDF24Doc.prototype.setEmailCharset = function(charset) {
    this.setParam('emailCharset', charset);
};

function PDF24Element() {
    this.params = new Object();
}
PDF24Element.prototype.setParam = function(paramKey, paramValue) {
    this.params[paramKey] = paramValue;
};
PDF24Element.prototype.setTitle = function(title) {
    this.setParam('title', title);
}
```

Extracto de la mencionada librería, con funciones para establecer ciertos atributos y características al archivo generado, con capacidad de trabajar con e-mails

6.2.6 CREACIÓN DE MENSAJES DE AVISO

Muchas de las acciones que se pueden realizar en esta web se notifican al usuario como un mensaje de éxito o error que aparece momentáneamente en la parte superior de la pantalla al realizarlas. Todos los mensajes están almacenados en un fichero Javascript llamado “popup_box_create.js” El funcionamiento general se basa en la redirección por parte de los controladores a las secciones de la web donde se realizan las acciones invocadas, pero añadiendo un nuevo parámetro a la queryString de la URL con valores como 1, 2, 0, -1... que determinan el mensaje que muestra el aviso.

```
function showBoxSuccessLogin(bool, name){
    const warning = document.createElement("div");

    if(bool==0){
        warning.textContent="Sesión cerrada correctamente.";
        warning.setAttribute("class", "warning alert alert-danger");
    }else if(bool==1){
        warning.textContent="Bienvenido " + name + ", has iniciado sesión correctamente.";
        warning.setAttribute("class", "warning alert alert-success");
    }else if(bool==2){
        warning.textContent="Datos actualizados correctamente, " + name;
        warning.setAttribute("class", "warning alert alert-success");
    }

    showBoxAux(warning);
};

////////////////////////////////////

function showBoxSuccessMember(bool){
    const warning = document.createElement("div");

    if(bool==1){
        warning.textContent="Suscripción comprada correctamente.";
        warning.setAttribute("class", "warning alert alert-success");
    }else if(bool==2){
        warning.textContent="Suscripción extendida correctamente.";
        warning.setAttribute("class", "warning alert alert-success");
    }else if(bool==-1){
        warning.textContent="Error al comprar la suscripción.";
        warning.setAttribute("class", "warning alert alert-danger");
    }else if(bool==-2){
        warning.textContent="Error al extender la suscripción.";
        warning.setAttribute("class", "warning alert alert-danger");
    }

    showBoxAux(warning);
};
```

Extracto del fichero mencionado mostrando dos funciones: La primera se ejecuta en el index de la página, avisando de inicio o cierre de sesión o edición correcta de datos. La segunda se ejecuta en la sección de gestión de suscripciones.

```
function showBoxAux(box){
    document.body.appendChild(box);

    setTimeout(function() { box.classList.add("show-warning"); }, 10);
    setTimeout(function() { box.classList.remove("show-warning"); }, 2000);
    setTimeout(function() { box.style.display = "none"; }, 2500);
}
```

Función de generación y ocultación del mensaje

```
.warning {
  position: fixed;
  top: -50px;
  left: 50%;
  transform: translateX(-50%);
  padding: 10px;
  margin-top: 15px;
  font-size: 16px;
  border-radius: 5px;
  z-index: 1000;
  width: 80%;
  text-align: center;
  transition: top 0.5s ease-in-out;
}
.show-warning { top: 0; }
```

Estilos necesarios para el correcto funcionamiento de los avisos



Ejemplo de funcionamiento #1: Inicio de Sesión



Ejemplo de funcionamiento #2: Comprar suscripción

6.2.7 CREACIÓN DE TABLAS PAGINADAS MEDIANTE DATATABLES

Gracias a la librería de Javascript 'DataTables', podemos generar de manera sencilla tablas en las que no solo se mostrarán los datos, sino que además cuentan con sus propios botones de paginación, ordenado de datos según el campo elegido o filtrado de estos.

Además de incluir los scripts y CDNs de JS y CSS de DataTable en el código, las tablas piden como requisito una cabecera (que deberá tener el mismo número de celdas que columnas a incluir) y un cuerpo.

```
/*-----USER LIST-----*/
echo "<a id='add-user' class='btn btn-log btn-shape element-green-bg'>Agregar Usuario</a>";
echo "<table id='boxContent' style='font-size: 0.9em;' class='table table-striped'><thead><tr><th>ID</th><th>E-Mail</th><th>CIF</th><th>Nombre</th>
<th>Teléfono</th><th>Provincia</th><th>Fecha_Registro</th><th>Tipo</th><th>Acciones</th></tr></thead>";
echo "<tbody></tbody></table>";
/*-----USER LIST-----*/
<script>
var userControl = <?php echo json_encode($userControl); ?>;

document.addEventListener('DOMContentLoaded', function () {
    new DataTable('#boxContent', {
        data: userControl,
        columns: [
            { data: 'Usuario_ID', render: function(data, type, row) {
                var btnId = (row["Estado"]==1) ? "enabled-" : "disabled-";
                return "<a href='#' id='"+btnId+data+"' onclick='activeUser(\"+data+\")'+data+'</a>";
            }
        },
            { data: 'Email' },
            { data: 'CIF' },
            { data: 'Nombre', render: function(data, type, row) {
                return "<a href='principal.php?methodUser=viewProfile&id="+row['Usuario_ID']+"'+data+'</a>";
            }
        },
            { data: 'Telefono' },
            { data: 'Provincia' },
            { data: 'Fecha_Registro', render: function(data, type, row) { return new Date(data).toLocaleDateString(); } },
            { data: 'Tipo', render: function(data, type, row) {
                if (data === "C") return "Cliente";
                if (data === "P") return "Proveedor";
                return "No Identificado";
            }
        },
            { data: null, render: function(data, type, row) {
                return "<a href='#' class='has-tooltip' data-tooltip='Actualizar Usuario' onclick='updateUser(\"+row[\"Usuario_ID\"]+\")'+<i class='f
                <a href='#' class='has-tooltip' data-tooltip='Eliminar Usuario' onclick='deleteUser(\"+row[\"Usuario_ID\"]+\")'+<i class='fa-solid f
            }
        ],
        scrollX: true
    });
});
</script>
```

En el código superior se puede ver cómo se construye la tabla: se lleva los datos de la consulta al script mediante la función `json_encode()`, y se inicializa la `DataTable` en la tabla de ID 'boxContent', que contiene una cabecera con 9 celdas y un cuerpo.

Dentro de las columnas, cada valor de 'data' representa las diferentes claves del array (u objeto) que contiene los datos de la consulta. Mediante la función 'render', podemos imprimir información más allá del simple valor contenido. Por ejemplo, en el caso de la columna 'Nombre', no solo se devuelve el nombre del usuario (parámetro 'data') sino que se le asigna un enlace que lleva a la página de su perfil (utilizando su ID como variable en la queryString del enlace, 'row [Usuario_ID]').

Showing 1 to 10 of 20 entries

Search:

ID	E-Mail	CIF	Nombre	Teléfono	Provincia	Fecha_Registro	Tipo	Acciones
1	proveedor1@mail.com	A00000001	Proveedor 1	100000001	Huelva	11/30/2025	Proveedor	 
2	proveedor2@mail.com	A00000002	Proveedor 2	100000002	Huelva	11/30/2025	Proveedor	 
3	proveedor3@mail.com	A00000003	Proveedor 3	100000003	Huelva	11/30/2025	Proveedor	 
4	proveedor4@mail.com	A00000004	Proveedor 4	100000004	Huelva	11/30/2025	Proveedor	 
5	proveedor5@mail.com	A00000005	Proveedor 5	100000005	Huelva	11/30/2025	Proveedor	 
6	proveedor6@mail.com	A00000006	Proveedor 6	100000006	Huelva	11/30/2025	Proveedor	 
7	proveedor7@mail.com	A00000007	Proveedor 7	100000007	Huelva	11/30/2025	Proveedor	 
8	proveedor8@mail.com	A00000008	Proveedor 8	100000008	Huelva	11/30/2025	Proveedor	 
9	proveedor9@mail.com	A00000009	Proveedor 9	100000009	Huelva	11/30/2025	Proveedor	 
10	proveedor10@mail.com	A00000010	Proveedor 10	100000010	Huelva	11/30/2025	Proveedor	 

Showing 1 to 10 of 20 entries

Previous 1 2 Next

Vistazo final a la DataTable creada, mostrando botones y enlaces creados mediante la función 'render:'.

Nótese además que cada tabla contiene un selector de entradas para su paginación, un formulario de filtrado y botones en cada columna de la cabecera para ordenar la tabla según el campo deseado.

6.2.8 PAGINACIÓN DE CONTENIDO SIN TABLAS

Existe contenido que no puede ser mostrado mediante DataTables, como tarjetas de productos o reseñas sobre estos. Para poder mostrarlos de manera paginada de forma sencilla, se ha optado por usar un método similar mediante Javascript, aplicando la función `json_encode()` a la consulta con los datos.

```
<div id='boxContent' class='row card-deck'></div>
<!--NAV BUTTONS-->
<?php
    $class0=$class1=$class2="btn btn-log element-green-bg ";
    $class1=$class0."not-visible";
    if(count($rentControl)<=8) $class2=$class0."not-visible";

    echo "<div class='nav-buttons site-article'>";
    echo "<div class='btn-group'><a class='$class1' id='btn-prev' href='#'>Anterior</a></div>";
    echo "<div class='btn-group'><a class='$class0' id='btn-page' href='#'>1</a></div>";
    echo "<div class='btn-group'><a class='$class2' id='btn-next' href='#'>Siguiete</a></div>";
    echo "</div>";
?>
<!--NAV BUTTONS-->
```

Div donde se mostrarán las tarjetas de los productos junto a los botones de navegación

```

function content_paginate(contentList, itemAmount){
    //////////////////////////////////////////////////CARGA INICIAL////////////////////////////////////
    $("#boxContent").on("load", function(){
        for(let i=0; i<itemAmount; i++){
            if(contentList[i]!=undefined)
                createContent(contentList[i]);
            else break;

            if(contentList[i+1]==undefined) $("#btn-next").addClass("not-visible");
            else $("#btn-next").removeClass("not-visible");
        }
    });
    $("#boxContent").trigger("load");
    //////////////////////////////////////////////////CARGA INICIAL////////////////////////////////////

    //////////////////////////////////////////////////BOTÓN DERECHO////////////////////////////////////
    $("#btn-next").on("click", function(){
        $("#boxContent").empty().fadeOut(100).fadeIn(100);
        let page = parseInt($("#btn-page").text())+1;
        let offset = (page-1)*itemAmount;

        for(let i=offset; i<offset+itemAmount; i++){
            if(contentList[i]!=undefined)
                createContent(contentList[i]);
            else break;
            if(contentList[i+1]==undefined) $("#btn-next").addClass("not-visible");
            else $("#btn-next").removeClass("not-visible");
        }
        $("#btn-prev").removeClass("not-visible");
        $("#btn-page").text(page);

        event.preventDefault();
    });
    //////////////////////////////////////////////////BOTÓN DERECHO////////////////////////////////////

    //////////////////////////////////////////////////BOTÓN IZQUIERDO////////////////////////////////////
    $("#btn-prev").on("click", function(){
        $("#boxContent").empty().fadeOut(100).fadeIn(100);
        let page = parseInt($("#btn-page").text())-1;
        let offset = (page-1)*itemAmount;

        for(let i=offset; i<offset+itemAmount; i++){
            if(contentList[i]!=undefined)
                createContent(contentList[i]);
            else break;
        }
        if(page==1) $("#btn-prev").addClass("not-visible");
        else $("#btn-prev").removeClass("not-visible");
        $("#btn-next").removeClass("not-visible");
        $("#btn-page").text(page);

        event.preventDefault();
    });
    //////////////////////////////////////////////////BOTÓN IZQUIERDO////////////////////////////////////

```

Función de paginación. Recibe el objeto y la cantidad de productos mostrados y, dependiendo de la página y/o de la cantidad mostrada, oculta los botones de Anterior o Siguiente.

```

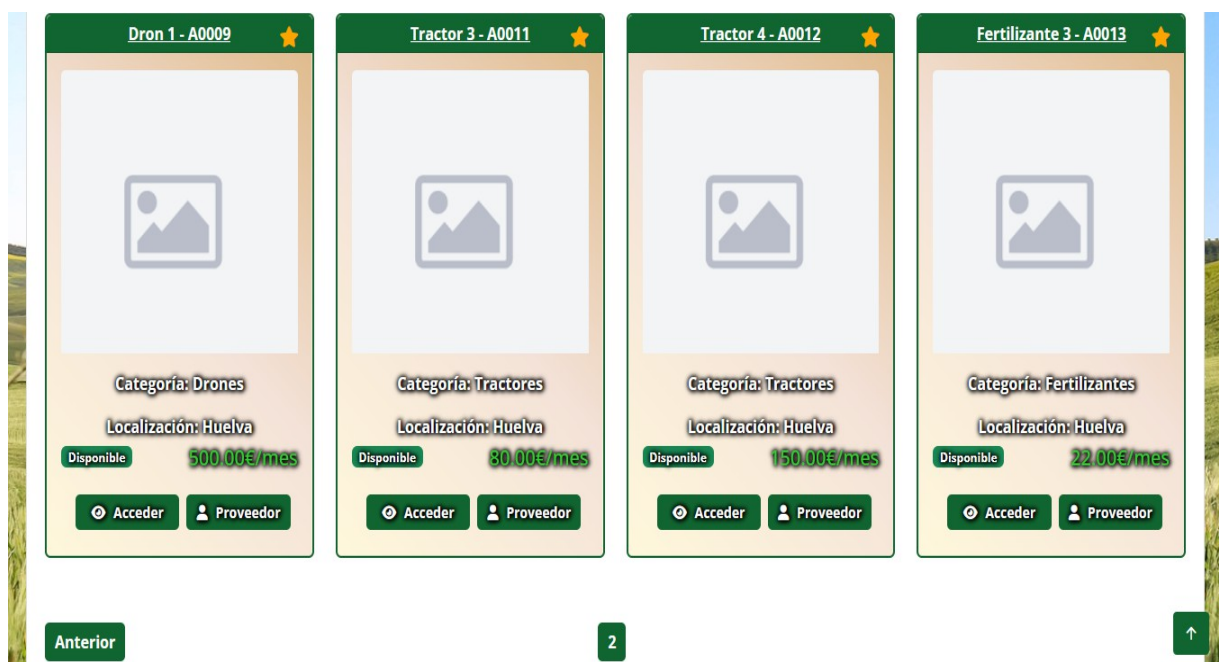
function createContent(rent){
  let file = (rent["Imagen"]!=null) ? rent["Imagen"] : "anon.png";
  let deuda = (rent["Deuda"]!=null) ? rent["Deuda"] : 0 ;
  let buttons = "";

  if(rent['Deuda']>0){
    buttons += "<div class='col-12 d-flex gap-2'" +
      "<a href='#' class='btn btn-sm btn-product-modify btn-shape w-50' onclick='insertPay(\"+rent['RID']+\", \"+rent['PID']+\", \"+rent['Deuda']+\")'" +
      "<a href='#' class='btn btn-sm element-green-bg btn-shape w-50' onclick='viewPay(\"+rent['RID']+\")'><i class='fa-solid fa-credit-card bor" +
    "</div>";
  }else{
    buttons += "<div class='col-12 d-flex gap-2'" +
      "<a href='#' class='btn btn-sm btn-product-extend btn-shape w-33' onclick='insertPay(\"+rent['RID']+\", \"+rent['PID']+\", \"+rent['Deuda']+\")'" +
      "<a href='#' class='btn btn-sm btn-product-delete btn-shape w-33' onclick='activeRent(\"+rent['RID']+\", \"+rent['PID']+\", \"+rent['Nombre']+\")'" +
      "<a href='#' class='btn btn-sm element-green-bg btn-shape w-33' onclick='viewPay(\"+rent['RID']+\")'><i class='fa-solid fa-credit-card bor" +
    "</div>";
  }

  $("#boxContent").append(
    "<div id='rent-"+rent['RID']+"' class='col-12 col-md-6 col-lg-4'" +
    "<div class='card h-100 element-green-border'" +
    "<div class='card-header element-green-bg'" +
    "<div class='card-body d-flex flex-column'" +
    "<img src='../assets/img/products/'+file+'" class='card-img-top mb-3' alt='"+rent["Nombre"]+"'> +
    "<p class='card-text mb-1'>Precio Total: <strong style='color:limegreen'>"+rent['Precio_Total']+"€</strong></p> +
    "<p class='card-text mb-1'>Cantidad Pendiente: <strong style='color:limegreen'>"+deuda+"€</strong></p> +
    "<p class='card-text mb-1'>Fecha Inicio: <span>"+(rent["Fecha_Inicio"] ? new Date(rent["Fecha_Inicio"]).toLocaleDateString() : ' ')" +
    "<p class='card-text mb-1'>Fecha Fin: <span>"+(rent["Fecha_Fin"] ? new Date(rent["Fecha_Fin"]).toLocaleDateString() : ' ')" +
    "<div class='mt-auto'" + buttons + "</div> +
    "</div> +
    "</div> +
    "</div>"
  );
}

```

Función 'createContent' para crear las tarjetas de los productos alquilados por el usuario. Se determinan primero ciertos valores como la lista de botones a mostrar (asignándoles las funciones correspondientes), y después se construye la tarjeta completa en el div principal.



Vista de la página de productos Favoritos. Habiendo marcado 12 como tal y mostrando 8 a la vez.

7. MANUAL DE USO

7.1 REQUISITOS E INSTALACIÓN

Para instalación de esta aplicación, es necesario un PC con al menos el Sistema Operativo Windows 7, y haber instalado la aplicación XAMPP. Tras esto, moveremos la carpeta del proyecto a C:\xampp\htdocs. Inicializamos XAMPP y activamos Apache y MySQL. Finalmente, abrimos nuestro navegador y en la barra de navegación vamos a 'localhost/farmxpress/index.php'.

7.2 FUNCIONES USUARIO INVITADO

- **REGISTRARSE:** Crear una cuenta en la aplicación. Como se menciona en apartados anteriores, los datos a rellenar son el E-Mail, CIF, Nombre, Contraseña, Dirección y Teléfono de la empresa, además de seleccionar Comunidad Autónoma, Provincia (cuyo selector cambia automáticamente según la CA elegida), y (opcional) subir una imagen de perfil. En este caso, sólo se puede registrar como Cliente.
- **CONSULTAR CATÁLOGO:** El usuario invitado puede echar un vistazo al catálogo completo de productos, pero no podrá entrar a la ficha del producto para alquilarlo a menos que inicie sesión en primer lugar.
- **INICIAR SESIÓN:** Entrar en la aplicación como usuario, utilizando el Nombre y la contraseña.

Registro de Usuario

Email: *

ejemplo@gmail.com

Nombre: * CIF: *

Nombre de Empresa Ejemplo de CIF: A12345678

Teléfono: * Dirección: *

Ejemplo de teléfono: 123456789 Ejemplo de Dirección

Comunidad Autónoma: * Provincia: *

Seleccione Seleccione

Contraseña: * Avatar:

Ejemplo de contraseña Browse... No file selected.

PNG o JPG de máximo 1MB

☐ Acepto los Términos y Condiciones y la Política de Privacidad

Enviar

Los campos marcados con un * son obligatorios

Visualización del formulario de registro

7.3 FUNCIONES USUARIO CLIENTE

- **ACTUALIZAR DATOS:** Modificar datos de la cuenta, excepto la contraseña.
- **CONSULTAR CATÁLOGO:** Ver el listado de productos existentes en la página. Se puede filtrar por categoría, precio mínimo y máximo, Comunidad Autónoma y Provincia. Al igual que en el formulario de registro, el contenido del selector de provincia cambia automáticamente según la CA elegida.
- **VER PERFIL PROVEEDOR:** Es posible acceder al perfil del proveedor del producto, en donde el cliente podrá ver su información de contacto y el resto de sus productos.
- **ALQUILAR PRODUCTO:** Acceder a la ficha de datos de un producto y, si está disponible, elegir el número de meses durante los que se alquilará el producto.
- **DESCARGAR FACTURA:** Descargar una factura en formato PDF con los datos del alquiler tras realizar el pago inicial mediante Stripe.
- **MARCAR FAVORITOS:** Añadir un producto a la sección de productos favoritos del usuario, para poder realizar un seguimiento más fácil de estos, o eliminarlos. Al pulsar en “Productos Seguidos”, se podrá ver todos estos productos.
- **GESTIONAR ALQUILERES:** Consultar todos los productos que se encuentren alquilados por el usuario en ese momento.
 - **REALIZAR PAGO:** Pagar parcial o totalmente la cantidad debida de un alquiler hasta la fecha, mediante la plataforma de pago Stripe.
 - **EXTENDER ALQUILER:** Junto al pago, opcionalmente se puede extender los meses de duración del alquiler. Si no debes dinero por el alquiler en cuestión, el botón de pago se sustituye por el de “Extender Duración”, el cual elimina el campo de selección de cantidad y la opción de no extenderlo.
 - **FINALIZAR ALQUILER:** Si nuestra deuda es nula, podremos finalizar el alquiler, y el producto volverá a estar disponible para que lo alquile otro usuario.
- **ESCRIBIR RESEÑA DE PRODUCTO:** Si ya hemos alquilado el producto en cuestión al menos una vez, podremos colgar una reseña en su página de detalles. Consiste en un texto y una calificación del 0 al 5, que podremos elegir haciendo click en las estrellas vacías situadas encima del área de texto. Estas estrellas cambiarán de estilo, rellenando su interior para mostrar la calificación marcada.
- **ELIMINAR RESEÑA:** Eliminar nuestra reseña localizándola y haciendo click en el botón Eliminar. Tras ello, podremos escribir una nueva reseña.
- **ADQUIRIR MEMBRESÍA:** Es posible comprar una de las suscripciones existentes haciendo click en el botón “Comprar” de la tarjeta de la suscripción deseada, y realizando el pago inicial correspondiente al primer mes.
- **EXTENDER MEMBRESÍA:** Extender duración de la membresía.

7.4 FUNCIONES USUARIO PROVEEDOR

- **ACTUALIZAR DATOS:** Modificar datos de la cuenta, excepto la contraseña.
- **GESTIONAR PRODUCTOS:** Ver el listado de los productos que hayamos insertado en la web.
 - **VER ESTADÍSTICAS:** Ver estadísticas de un producto como los usuarios que lo han alquilado a lo largo del tiempo, los ingresos totales generados y la calificación media.
 - **INSERTAR PRODUCTO:** Invocar al formulario de subida de productos en un modal. Se debe rellenar el nombre, descripción, código de referencia y precio mensual de alquiler del producto, y, si se desea, subir una imagen.
 - **ACTUALIZAR PRODUCTO:** Modificar los datos del producto deseado.
 - **ELIMINAR PRODUCTO:** Borrar el producto deseado. Sólo se podrá eliminar si está disponible, es decir, si nadie lo ha alquilado.
- **ADQUIRIR MEMBRESÍA:** Es posible comprar una de las suscripciones existentes haciendo click en el botón “Comprar” de la tarjeta de la suscripción deseada, y realizando el pago inicial correspondiente al primer mes.
 - **EXTENDER MEMBRESÍA:** Extender los meses de duración de la membresía adquirida.

7.5 FUNCIONES ADMINISTRADOR

Para acceder como usuario Administrador, es necesario introducir las siguientes credenciales al iniciar sesión: “ADMINISTRADOR” como nombre de usuario, y “ADMINAPP” como contraseña. Al entrar, veremos este panel de administración:



Las funciones que podemos utilizar pulsando los botones de la barra lateral (o los botones de Más Información) son los siguientes:

- **GESTIONAR USUARIOS:** Ver datos de todos los usuarios en una tabla paginada (10 usuarios por página), y modificar sus datos (botón de engranaje) o eliminarlo (botón de papelera). Al pulsar el botón “Añadir Usuario”, podremos crear manualmente un usuario. Es la única forma de crear Proveedores.
- **GESTIONAR CATEGORÍAS:** Ver información paginada de las diferentes categorías, modificar sus datos, eliminarlas o añadir nuevas.
- **GESTIONAR SUSCRIPCIONES:** Ver información paginada de las diferentes suscripciones, modificar sus datos, eliminarlas o añadir nuevas. Al pulsar el botón de plus, podremos añadir ventajas a la suscripción deseada, y ver el listado de ventajas al pulsar el botón de ojo.

Suscripción ID	Nombre	Precio Mensual	Duración Base	Acciones												
1	Básica	10.00 €	1 meses	[+][🔍][⚙️][🗑️]												
<table border="1"> <thead> <tr> <th>Ventaja ID</th> <th>Nombre</th> <th>Descripción</th> <th>Acciones</th> </tr> </thead> <tbody> <tr> <td>1</td> <td>Básico 1</td> <td>Acceso básico</td> <td>[+][🗑️]</td> </tr> <tr> <td>2</td> <td>Básico 2</td> <td>Soporte limitado</td> <td>[+][🗑️]</td> </tr> </tbody> </table>					Ventaja ID	Nombre	Descripción	Acciones	1	Básico 1	Acceso básico	[+][🗑️]	2	Básico 2	Soporte limitado	[+][🗑️]
Ventaja ID	Nombre	Descripción	Acciones													
1	Básico 1	Acceso básico	[+][🗑️]													
2	Básico 2	Soporte limitado	[+][🗑️]													
2	Pro	25.00 €	3 meses	[+][🔍][⚙️][🗑️]												
3	Premium	40.00 €	6 meses	[+][🔍][⚙️][🗑️]												

Interfaz de gestión de Suscripciones y Ventajas

- **GESTIONAR PRODUCTOS:** Ver el catálogo de productos y acceder a ellos.
 - **GESTIONAR RESEÑAS:** Es posible acceder a la ficha de cada producto y eliminar todas las reseñas que el Administrador considere inapropiadas. Por obvias razones, NO es posible realizar un alquiler del producto.
- **GESTIONAR LOGS:** Se ven las siguientes opciones al pulsar “Gestionar Logs”:
 - **VISUALIZAR DATOS:** Podremos ver el tráfico que recibe la web o información de los alquileres realizados. Podemos filtrar la información por usuario, y elegir una fecha de inicio y/o una fecha límite.

- **IMPRIMIR LOG:** Una vez recibida y mostrada toda la información, es posible crear un fichero TXT con los datos automáticamente al pulsar “Imprimir Informe”. Los ficheros se nombran según el tipo (Visitas o Alquileres) y por fecha de creación.
- **LEER LOG:** Si hemos creado logs de información con anterioridad, podemos leer los ficheros existentes al pulsar “Consultar Logs Existentes” y elegir el deseado.

Información sobre el tráfico cargada correctamente

[Imprimir Informe](#)

Show entries

Search:

Usuario ID	Nombre Usuario	CIF	Nombre Producto	Fecha Inicio	Fecha Fin	Precio Total	Estado
11	Cliente 1	B00000001	Tractor 1	1/1/2023	11/30/2025	120.00	Finalizado
11	Cliente 1	B00000001	Tractor 3	1/1/2023	11/30/2025	80.00	Finalizado
12	Cliente 2	B00000002	Tractor 2	2/1/2023	11/30/2025	130.00	Finalizado
12	Cliente 2	B00000002	Tractor 4	2/1/2023	11/30/2025	150.00	Finalizado
13	Cliente 3	B00000003	Fertilizante 1	3/1/2023	11/30/2025	20.00	Finalizado
13	Cliente 3	B00000003	Fertilizante 3	3/1/2023	11/30/2025	22.00	Finalizado
14	Cliente 4	B00000004	Fertilizante 2	4/1/2023	11/30/2025	25.00	Finalizado
14	Cliente 4	B00000004	Fertilizante 4	4/1/2023	11/30/2025	26.00	Finalizado
15	Cliente 5	B00000005	Pala 1	5/1/2023	11/30/2025	15.00	Finalizado

Interfaz de visualización de datos

8. CONCLUSIONES

El objetivo de FamXPress es ofrecer una solución eficaz para el alquiler de maquinaria agrícola en España, mediante una plataforma que permita la oferta de los productos por parte de los Proveedores autorizados, debido a la carencia de una página libre y especializada para ello.

La integración de tecnologías como Stripe permite realizar pagos de forma segura, y el uso del patrón de programación MVC permite tener un código estructurado, organizado, escalable y fácilmente mantenible al prevenir el llamado “código espagueti”. Por otro lado, tecnologías front-end como JQuery o AJAX permite el manejo de datos en tiempo real, como el seguimiento de productos, haciendo más cómoda la experiencia para el usuario.

Además, el diseño a partir de una plantilla de Bootstrap ha permitido construir una interfaz amigable y responsiva de forma sencilla, tanto para usuarios comunes como para usuarios administradores.

En resumen, la aplicación propuesta se presenta como una idea con potencial para mejorar y sentar un referente en el sector agrícola español, ayudando a empresarios a ofrecer y alquilar maquinaria mediante una web de uso sencillo pero eficaz.

9. RECURSOS Y FINANCIACIÓN

Para el desarrollo de este proyecto se usará el siguiente [repositorio de GitHub](#), en el que se publicarán commits y ramas semanales con cada nueva función o avance.

Además, se ha escogido como hosting la web InfinityFree, un hosting gratuito de PHP y MySQL sin anuncios que permite el envío de correos mediante cualquier SMTP (en el caso de esta aplicación, Gmail mediante PHPMailer). Se han investigado diferentes hostings y se ha recabado esta información:

Hosting	Almacenamiento	Ancho de Banda	Anuncios?	Envío de emails
InfinityFree	5GB	Ilimitado	Sin anuncios forzados	Permite envío de correos con un SMTP Externo
ProFreeHost	5GB	Ilimitado	Sin anuncios forzados	No se ha podido encontrar suficiente información

El presente proyecto no requiere una inversión económica elevada, ya que se ha desarrollado utilizando principalmente recursos propios y herramientas de software libre, como un ordenador propio, Visual Studio Code, Github, XAMPP... En cuanto al hosting escogido, se usará InfinityFree con un plan gratuito, aunque si se necesitara, se podría cambiar a un plan premium, que cuesta 10€ mensuales.

10. FUTURO DE LA APLICACIÓN

Ideas cuya implementación podría resultar interesante en el futuro:

- **MENSAJES PRIVADOS:** Dar la posibilidad a los clientes y proveedores comunicarse entre ellos mediante un buzón de mensajes dentro de la aplicación.
- **FORMULARIO DE CONTACTO PARA EL ADMINISTRADOR:** Crear un formulario de contacto que permita al administrador enviar un correo electrónico a cualquier usuario de forma cómoda.

11. ENLACES EXTERNOS

- **AGRICULTURE:** <https://bootstrapmade.com/agriculture-bootstrap-website-template/>
- **ADMINLTE:** <https://adminlte.io/>
- **DOCUMENTACIÓN JQUERY:** <https://api.jquery.com/>
- **DOCUMENTACIÓN PDF24:** <https://www.pdf24.org/es/>
- **DOCUMENTACIÓN DATATABLES:** <https://datatables.net/>
- **DOCUMENTACIÓN STRIPE:** <https://docs.stripe.com>
- **DOCUMENTACIÓN PHPMAILER:** <https://github.com/PHPMailer/PHPMailer>
- **GUÍA BOOTSTRAP:** <https://getbootstrap.com/docs/4.1/getting-started/introduction/>
- **ESQUEMA DE TRABAJO TRELLO:** <https://trello.com/b/3Ddl4nGY/my-trello-board>
- **REPOSITORIO GITHUB FARMXPRESS:** <https://github.com/ManuelJesus12/FarmXPress>
- **ENLACE A LA WEB ALOJADA EN INFINITYFREE:** farmxpress.infinityfree.me